

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng phần mềm điều khiển vận hành ROV -
Phân hệ quản lý thông tin nhiệm vụ**

Lê Minh Thành

thanhle20072004@gmail.com

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Lam Trung

Khoa: Công nghệ thông tin Việt Nhật

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 05/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng phần mềm điều khiển vận hành ROV -
Phân hệ quản lý thông tin nhiệm vụ**

Lê Minh Thành

thanhle20072004@gmail.com

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Lam Trung _____

Chữ kí GVHD

Khoa: Công nghệ thông tin Việt Nhật

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 05/2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn sâu sắc và chân thành nhất đến thầy **TS. Ngô Lam Trung** — giảng viên hướng dẫn của em. Trong suốt quá trình thực hiện đồ án, thầy đã không chỉ định hướng về mặt kỹ thuật và học thuật, mà còn luôn đồng hành, kiên nhẫn giải đáp từng thắc mắc và chỉ ra những điểm cần cải thiện một cách chi tiết. Sự tận tâm và kinh nghiệm thực tiễn của thầy đã giúp em hoàn thiện đồ án đúng hướng và đạt kết quả tốt hơn rất nhiều so với những gì em có thể tự làm.

Em xin chân thành cảm ơn Ban Giám hiệu Trường Đại học Bách khoa Hà Nội, Ban lãnh đạo Trường Công nghệ Thông tin và Truyền thông, cùng toàn thể các thầy cô trong Khoa Công nghệ thông tin Việt-Nhật đã tạo dựng môi trường học thuật nghiêm túc và trang bị cho em nền tảng kiến thức vững chắc trong suốt bốn năm đào tạo. Những kiến thức về kỹ thuật phần mềm, cơ sở dữ liệu, mạng máy tính và trí tuệ nhân tạo mà em được học là nền móng trực tiếp để thực hiện đồ án này.

Em xin bày tỏ lòng biết ơn đến gia đình — những người luôn là chỗ dựa tinh thần vững chắc, chấp nhận những đêm thức khuya và những giai đoạn em phải toàn tâm toàn ý với đồ án. Sự ủng hộ và tin tưởng của gia đình là động lực lớn nhất giúp em vượt qua những thời điểm khó khăn nhất của quá trình nghiên cứu.

Cuối cùng, em xin cam đoan đồ án tốt nghiệp này là công trình nghiên cứu độc lập của bản thân, được thực hiện dưới sự hướng dẫn của TS. Ngô Lam Trung. Toàn bộ mã nguồn, kết quả thực nghiệm và nội dung trình bày đều là kết quả lao động thực sự của em; các tài liệu tham khảo và trích dẫn đều được chú thích đầy đủ theo quy định.

Hà Nội, tháng 5 năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Lê Minh Thành

TÓM TẮT NỘI DUNG ĐỒ ÁN

Phương tiện lặn điều khiển từ xa (ROV) ngày càng được ứng dụng rộng rãi trong các nhiệm vụ khảo sát dưới nước như kiểm tra kết cấu công trình ngầm, quan trắc hệ sinh thái biển và thu thập dữ liệu địa chất. Trong quá trình vận hành thực tế, toàn bộ dữ liệu cảm biến (nhiệt độ, áp suất, độ sâu, định vị GPS) và phương tiện truyền thông (video, ảnh) được ghi lại trực tiếp trên thiết bị, sau đó người vận hành mới thực hiện tải lên hậu kỳ sau khi ROV về bờ do môi trường thực địa không có kết nối mạng ổn định. Tuy nhiên, quy trình phân tích sau chuyển lặn hiện nay hoàn toàn rời rạc: người phân tích phải mở đồng thời trình phát video và tệp CSV cảm biến, đối chiếu thủ công từng dòng dữ liệu với từng khung hình để xác định trạng thái môi trường tại một thời điểm cụ thể. Quy trình này không chỉ tốn nhiều thời gian và dễ sai sót, mà còn thiếu khả năng phân tích tự động và chia sẻ dữ liệu trong nhóm vận hành. Các giải pháp hiện có như QGroundControl hay BlueOS được thiết kế cho điều khiển thời gian thực, chưa đáp ứng nhu cầu quản lý và phân tích dữ liệu hậu kỳ trên nền tảng web đa người dùng.

Đồ án này đề xuất và xây dựng hệ thống web “*Quản lý Vận hành ROV tích hợp Trí tuệ Nhân tạo*” theo kiến trúc Client-Server đa tầng, lấy việc số hóa và tự động hóa quy trình phân tích hậu kỳ làm trọng tâm. Về mặt kỹ thuật, hệ thống áp dụng ngăn xếp công nghệ hiện đại bao gồm Node.js/Express.js cho tầng máy chủ, MongoDB cho lưu trữ dữ liệu cảm biến dạng tài liệu linh hoạt, React 18 với Vite cho giao diện người dùng tương tác, Redis cho hàng đợi tác vụ và bảo mật phiên đăng nhập. Điểm khác biệt cốt lõi so với các giải pháp hiện có là việc tích hợp ba mô-đun trí tuệ nhân tạo độc lập: mô-đun nhận diện vật thể sử dụng mô hình YOLOv8n chạy trên Python FastAPI microservice để tự động phát hiện và theo dõi vật thể trong từng khung hình video theo cơ chế bất đồng bộ qua Bull Queue; mô-đun tóm tắt nhiệm vụ sử dụng Gemini 2.5 Flash API để tổng hợp toàn bộ thông tin chuyển lặn thành văn bản báo cáo tự nhiên; và mô-đun phát hiện bất thường cảm biến triển khai thuật toán Z-Score không cần dữ liệu huấn luyện để tự động gán cờ các điểm đo bất thường. Hệ thống còn cung cấp tính năng đồng bộ video–biểu đồ cảm biến theo thời gian thực và hệ thống trích xuất bằng chứng (Evidence System) cho phép người vận hành chụp ảnh khung hình và đánh dấu đoạn clip trực tiếp trong khi xem video.

Hệ thống được đóng gói theo mô hình triển khai đề xuất sử dụng Docker Compose với bốn dịch vụ (nginx, node-backend, yolo-service, redis) trên VPS, giao diện frontend phân phối qua Vercel CDN, cơ sở dữ liệu trên MongoDB

Atlas và lưu trữ file trên AWS S3. Kết quả kiểm thử chức năng đạt 37/37 test case (100%), bao gồm các nhóm xác thực, phân quyền RBAC, kiểm tra đầu vào và CRUD toàn bộ tài nguyên. Kiểm thử tải bằng script Node.js tùy chỉnh ở mức 20 người dùng đồng thời cho thấy thời gian phản hồi trung bình dưới 500ms với p95 dưới 1 giây cho các endpoint dữ liệu thông thường; endpoint thống kê tổng hợp (dashboard) đạt p95 khoảng 1,8 giây do cần thực hiện bảy aggregation song song trên MongoDB. Mô hình YOLOv8n xử lý một ảnh đơn trong dưới một giây và video với tốc độ lấy mẫu thích nghi (0,2–1,0 giây mỗi khung hình tùy độ dài clip). Toàn bộ kết quả phân tích AI được đẩy về giao diện theo cơ chế Server-Sent Events (SSE) mà không cần tải lại trang.

Hệ thống đặt nền móng cho việc số hóa toàn diện quy trình vận hành và quản lý dữ liệu ROV, thay thế thao tác thủ công rời rạc bằng một nền tảng tập trung, đa người dùng và tự động hóa. Về hướng phát triển, hệ thống có thể được mở rộng tích hợp thêm luồng GCS thời gian thực khi có kết nối mạng ngoài thực địa, tinh chỉnh mô hình YOLOv8 trên tập dữ liệu ROV thực tế (san hô, mảnh vỡ, sinh vật biển) để nâng cao độ chính xác, hoặc phát triển ứng dụng di động cho người vận hành sử dụng ngay tại hiện trường. Khung kiến trúc microservice đã xây dựng tạo điều kiện thuận lợi cho việc bổ sung các mô hình AI chuyên biệt trong tương lai mà không cần thay đổi cấu trúc hệ thống.

Hà Nội, tháng 5 năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Lê Minh Thành

ABSTRACT

Remotely Operated Vehicles (ROVs) are increasingly deployed in underwater survey missions including the inspection of submerged structures, marine ecosystem monitoring, and geological data collection. In practice, all sensor data (temperature, pressure, depth, GPS positioning) and multimedia content (video, images) are recorded locally on the vehicle, and uploaded by operators only after the ROV returns to shore due to the absence of stable network connectivity in field environments. The post-dive analysis workflow currently relies entirely on manual cross-referencing: analysts must simultaneously open a video player and a CSV spreadsheet, manually correlating each data row with the corresponding video frame to determine environmental conditions at a specific moment. This approach is not only time-consuming and error-prone, but also lacks any automation capability and makes collaborative analysis within an operations team impractical. Existing solutions such as QGroundControl and BlueOS are designed primarily for real-time vehicle control and do not address the need for multi-user, web-based post-mission data management and analysis.

This thesis proposes and implements a web-based system titled “*AI-Integrated ROV Operations Management*” following a multi-tier Client-Server architecture, with the digitization and automation of post-mission analysis as its primary focus. The technology stack consists of Node.js/Express.js for the server tier, MongoDB for flexible document-oriented sensor data storage, React 18 with Vite for the interactive user interface, and Redis for task queue management and session security. The most distinctive aspect of the system compared to existing solutions is the integration of three independent artificial intelligence modules: an object detection module using the YOLOv8n model running on a Python FastAPI microservice, which automatically detects and tracks objects in video frames on a per-frame basis via an asynchronous Bull Queue pipeline; a mission summarization module leveraging the Gemini 2.5 Flash API to synthesize all dive information into a natural language report; and a sensor anomaly detection module implementing the Z-Score algorithm without requiring any labeled training data to automatically flag abnormal sensor readings. The system additionally provides real-time video-to-sensor-chart synchronization and an Evidence System that allows operators to capture frame snapshots and mark video clip segments directly while reviewing footage.

The system follows a proposed deployment model using Docker Compose

with four services (nginx, node-backend, yolo-service, redis) on a VPS, with the frontend distributed via Vercel CDN, the database hosted on MongoDB Atlas, and file storage on AWS S3. Functional testing results show 37 out of 37 test cases passing (100%), covering authentication, role-based access control (RBAC), input validation, and full CRUD operations across all resources. Load testing using a custom Node.js script at 20 concurrent users demonstrated average response times below 500ms with a p95 below 1 second for standard data endpoints; the aggregation-heavy dashboard statistics endpoint achieved a p95 of approximately 1.8 seconds owing to seven parallel MongoDB aggregation pipelines. The YOLOv8n model processes a single image in under one second and handles video with adaptive frame sampling (0.2–1.0 seconds per sample depending on clip duration). All AI analysis results are pushed to the client interface via Server-Sent Events (SSE) without requiring page reloads, providing a seamless real-time experience for the operator.

The system establishes a foundation for the comprehensive digitization of ROV operations and data management workflows, replacing fragmented manual processes with a centralized, multi-user, and automated platform. In terms of future development, the system can be extended to integrate a real-time Ground Control Station (GCS) stream when network connectivity is available in the field, to fine-tune the YOLOv8 model on domain-specific ROV datasets (coral reefs, debris classification, marine species) for improved detection accuracy, or to develop a companion mobile application for on-site operator use. The microservice architecture already in place provides a natural extension point for incorporating additional specialized AI models without restructuring the existing system.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	4
2.1 Khảo sát hiện trạng	4
2.2 Tổng quan chức năng	5
2.2.1 Biểu đồ use case tổng quát	6
2.2.2 Biểu đồ use case phân rã Phân hệ Dữ liệu Thực địa	7
2.2.3 Biểu đồ use case phân rã Phân hệ AI Phân tích	8
2.2.4 Quy trình nghiệp vụ tải lên và phân tích dữ liệu	9
2.3 Đặc tả chức năng	9
2.3.1 UC-01: Tải lên và đồng bộ dữ liệu cảm biến	12
2.3.2 UC-02: Kích hoạt phân tích nhận diện vật thể bằng YOLOv8.....	12
2.3.3 UC-03: Trích xuất và phân tích bằng chứng	13
2.3.4 UC-04: Tạo báo cáo tóm tắt bằng ngôn ngữ tự nhiên	14
2.4 Yêu cầu phi chức năng	15
2.5 Sơ đồ trạng thái	15
2.5.1 Vòng đời Chuyển khảo sát (Project)	16
2.5.2 Vòng đời Lướt lặn (Trip).....	17
2.5.3 Vòng đời phân tích Media	17

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	19
3.1 Nền tảng Giao diện Người dùng (Frontend).....	19
3.1.1 React 18 và Vite.....	19
3.1.2 Quản lý trạng thái: Zustand và React Query.....	20
3.1.3 Recharts và Leaflet.js	21
3.2 Nền tảng Máy chủ và Lưu trữ (Backend).....	21
3.2.1 Node.js và Express.js.....	21
3.2.2 MongoDB Atlas và Mongoose	21
3.2.3 AWS S3 và cơ chế Presigned URL.....	22
3.2.4 Xác thực JWT, Blacklist Redis và Google OAuth	22
3.3 Xử lý bất đồng bộ và Giao tiếp thời gian thực	23
3.3.1 Bull Queue và cơ chế xử lý bất đồng bộ	23
3.3.2 Server-Sent Events (SSE) và cơ chế đẩy thông báo	24
3.4 Hệ thống Trí tuệ Nhân tạo	25
3.4.1 YOLOv8 và Python Microservice.....	25
3.4.2 Gemini 2.5 Flash và AI Summary.....	26
3.4.3 Phát hiện bất thường Z-Score	26
3.4.4 Reverse Geocoding với OpenStreetMap Nominatim	26
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ	
HỆ THỐNG	28
4.1 Thiết kế kiến trúc hệ thống	28
4.1.1 Kiến trúc tổng quan.....	28
4.1.2 Sơ đồ phụ thuộc module	28
4.1.3 Mô hình triển khai đề xuất	29
4.2 Thiết kế cơ sở dữ liệu	30

4.3 Hiện thực các chức năng chính.....	31
4.3.1 Xác thực và phân quyền.....	31
4.3.2 Dashboard và thống kê tổng quan	32
4.3.3 Quản lý Project và Trip.....	33
4.3.4 Giao diện Cockpit TripDetailPage	35
4.3.5 Nhận diện vật thể YOLOv8 và Evidence System.....	36
4.3.6 Tổng hợp báo cáo bằng AI (Gemini)	38
4.3.7 Thông báo realtime và quản lý người dùng	38
4.3.8 Sơ đồ tuần tự các luồng nghiệp vụ chính	40
4.4 Kiểm thử hệ thống	43
4.4.1 Kiểm thử chức năng (Functional Test).....	43
4.4.2 Kiểm thử tải (Load Test).....	45
4.4.3 Kiểm thử hiệu năng Lighthouse.....	45
4.5 Mô hình triển khai (đề xuất)	46
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	48
5.1 Giải pháp đồng bộ hóa Video và Dữ liệu cảm biến thời gian thực.....	48
5.1.1 Bài toán đặt ra	48
5.1.2 Giải pháp hiện thực	48
5.1.3 Kết quả đạt được	49
5.2 Kiến trúc xử lý bất đồng bộ tích hợp mô hình nhận diện YOLOv8.....	49
5.2.1 Bài toán đặt ra	49
5.2.2 Giải pháp hiện thực	49
5.2.3 Kết quả đạt được	50
5.3 Hệ thống trích xuất bằng chứng (Evidence System)	50
5.3.1 Bài toán đặt ra	50
5.3.2 Giải pháp hiện thực	50

5.3.3 Kết quả đạt được	51
5.4 Phát hiện bất thường cảm biến tự động (Z-Score)	51
5.4.1 Bài toán đặt ra	51
5.4.2 Giải pháp hiện thực	51
5.4.3 Kết quả đạt được	52
5.5 Ứng dụng LLM tự động hóa báo cáo tổng kết nhiệm vụ	53
5.5.1 Bài toán đặt ra	53
5.5.2 Giải pháp hiện thực	53
5.5.3 Kết quả đạt được	53
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	54
6.1 Kết luận	54
6.2 Hướng phát triển.....	54
TÀI LIỆU THAM KHẢO.....	58
PHỤ LỤC.....	60
A. ĐẶC TẢ USE CASE.....	60
A.1 UC-B01: Đăng nhập bằng Email và Mật khẩu	60
A.2 UC-B02: Đăng nhập bằng tài khoản Google (OAuth 2.0).....	60
A.3 UC-B03: Quản lý Thiết bị ROV	61
A.4 UC-B04: Tạo và Quản lý Chuyến khảo sát (Project).....	62
A.5 UC-B05: Tạo Lướt lặn trong Chuyến khảo sát	62
A.6 UC-B06: Tải lên Tập phương tiện (Media Upload)	63
A.7 UC-B07: Quản lý Người dùng (Admin)	64
A.8 UC-B08: Xem Dashboard tổng quan.....	65

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ Use Case tổng quát của hệ thống	7
Hình 2.2	Biểu đồ Use Case phân rã Phân hệ Upload và Quản lý Dữ liệu Thực địa	8
Hình 2.3	Biểu đồ Use Case phân rã Phân hệ AI Phân tích	9
Hình 2.4	Biểu đồ hoạt động quy trình phân tích dữ liệu nhiệm vụ — Giai đoạn a: Thu thập và xử lý dữ liệu	10
Hình 2.5	Biểu đồ hoạt động quy trình phân tích dữ liệu nhiệm vụ — Giai đoạn b: Phân tích và tổng kết	11
Hình 2.6	Sơ đồ trạng thái vòng đời Chuyển khảo sát (Project)	16
Hình 2.7	Sơ đồ trạng thái vòng đời Lướt lặn (Trip)	17
Hình 4.1	Kiến trúc tổng quan hệ thống quản lý ROV	28
Hình 4.2	Sơ đồ phụ thuộc module phía backend	29
Hình 4.3	Mô hình triển khai đề xuất trên VPS	29
Hình 4.4	Sơ đồ ER cơ sở dữ liệu hệ thống quản lý ROV	30
Hình 4.5	Màn hình đăng nhập — hỗ trợ Google OAuth	32
Hình 4.6	Dashboard — KPI cards và biểu đồ thống kê tổng quan	33
Hình 4.7	Trang danh sách Projects — filter, search và export	34
Hình 4.8	ProjectDetailPage — TripCard expanded với DATA FILES và AI Summary	34
Hình 4.9	Trang danh sách Trips — filter theo status và date range	35
Hình 4.10	TripDetailPage Cockpit — layout 3 cột đầy đủ tính năng	35
Hình 4.11	Bottom chart tab Environment — dot đồ đánh dấu điểm bất thường Z-Score	36
Hình 4.12	Bounding box YOLO overlay trên ảnh/video ROV	36
Hình 4.13	AI Analyze Popover — chọn model và ngưỡng confidence trước khi phân tích YOLO	37
Hình 4.14	Evidence Panel — PHOTO/CLIP thumbnails và Evidence-Viewer overlay	38
Hình 4.15	AI Summary — báo cáo song ngữ được tổng hợp bởi Gemini 2.5 Flash	38
Hình 4.16	Notification bell dropdown — thông báo realtime qua SSE	39
Hình 4.17	Trang quản lý người dùng (Admin) — bulk operations và role management	40
Hình 4.18	Audit Log — nhật ký hoạt động admin theo timeline	40

Hình 4.19	Sơ đồ tuần tự — luồng tải lên file CSV cảm biến	41
Hình 4.20	Sơ đồ tuần tự — luồng phân tích YOLO bất đồng bộ qua Bull Queue	42
Hình 4.21	Kết quả kiểm thử chức năng — 37/37 ca đạt (100%)	44

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các giải pháp quản lý dữ liệu ROV hiện có	4
Bảng 2.2	Ma trận phân quyền RBAC	6
Bảng 2.3	Đặc tả Use Case UC-01: Tải lên và đồng bộ dữ liệu cảm biến .	12
Bảng 2.4	Đặc tả Use Case UC-02: Phân tích nhận diện vật thể bằng AI .	12
Bảng 2.5	Đặc tả Use Case UC-03: Trích xuất bằng chứng (Evidence System)	13
Bảng 2.6	Đặc tả Use Case UC-04: Tóm tắt nhiệm vụ bằng AI (AI Summary)	14
Bảng 2.7	Các trạng thái vòng đời phân tích Media	18
Bảng 3.1	Hệ sinh thái công nghệ của hệ thống	19
Bảng 3.2	Cấu hình các hàng đợi Bull Queue trong hệ thống	24
Bảng 3.3	Chiến lược lấy mẫu frame thích nghi của YOLO Microservice	25
Bảng 4.1	Mô tả các collection chính trong MongoDB	31
Bảng 4.2	Ma trận phân quyền RBAC của hệ thống	32
Bảng 4.3	Kết quả kiểm thử tải — 20 concurrent $\times$ 3 rounds (60 requests/endpoint)	45
Bảng 4.4	Kết quả kiểm thử Lighthouse — Desktop mode	45
Bảng A.1	Đặc tả Use Case UC-B01: Đăng nhập bằng Email/Mật khẩu .	60
Bảng A.2	Đặc tả Use Case UC-B02: Đăng nhập bằng Google OAuth 2.0	60
Bảng A.3	Đặc tả Use Case UC-B03: Quản lý Thiết bị ROV	61
Bảng A.4	Đặc tả Use Case UC-B04: Quản lý Chuyển khảo sát	62
Bảng A.5	Đặc tả Use Case UC-B05: Tạo Lướt lặn (Trip)	62
Bảng A.6	Đặc tả Use Case UC-B06: Tải lên Tập phương tiện qua Presigned URL	63
Bảng A.7	Đặc tả Use Case UC-B07: Quản lý Người dùng	64
Bảng A.8	Đặc tả Use Case UC-B08: Xem Dashboard tổng quan	65

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AI	Trí tuệ nhân tạo (Artificial Intelligence)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
AWS	Dịch vụ đám mây của Amazon (Amazon Web Services)
CDN	Mạng phân phối nội dung (Content Delivery Network)
CRUD	Tạo, Đọc, Cập nhật, Xóa (Create, Read, Update, Delete)
CSV	Giá trị phân tách bằng dấu phẩy (Comma-Separated Values)
DVL	Máy đo vận tốc Doppler (Doppler Velocity Log)
GPS	Hệ thống định vị toàn cầu (Global Positioning System)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
HTTP	Giao thức truyền siêu văn bản (HyperText Transfer Protocol)
JSON	Định dạng trao đổi dữ liệu nhẹ (JavaScript Object Notation)
JWT	Thẻ xác thực JSON (JSON Web Token)
KPI	Chỉ số hiệu suất then chốt (Key Performance Indicator)
LLM	Mô hình ngôn ngữ lớn (Large Language Model)
MSSV	Mã số sinh viên
NMS	Triệt tiêu cực đại không tối ưu (Non-Maximum Suppression)
OAuth	Giao thức ủy quyền mở (Open Authorization)
ODM	Ảnh xạ đối tượng–tài liệu (Object-Document Mapping)

Thuật ngữ	Ý nghĩa
ORM	Ánh xạ đối tượng–quan hệ (Object-Relational Mapping)
RBAC	Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control)
REST	Kiến trúc chuyển giao trạng thái đại diện (Representational State Transfer)
ROV	Phương tiện điều khiển từ xa dưới nước (Remotely Operated Vehicle)
S3	Dịch vụ lưu trữ đối tượng của Amazon (Simple Storage Service)
SEO	Tối ưu hóa công cụ tìm kiếm (Search Engine Optimization)
SPA	Ứng dụng đơn trang (Single Page Application)
SSE	Sự kiện do máy chủ đẩy (Server-Sent Events)
TTL	Thời gian sống của dữ liệu (Time To Live)
UI	Giao diện người dùng (User Interface)
URL	Định vị tài nguyên thống nhất (Uniform Resource Locator)
UX	Trải nghiệm người dùng (User Experience)
VPS	Máy chủ riêng ảo (Virtual Private Server)
YOLO	Thuật toán nhận diện vật thể thời gian thực (You Only Look Once)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương 1 giới thiệu tổng quan về bối cảnh nảy sinh đề tài và những thách thức đang gặp phải trong việc quản lý, vận hành thiết bị lặn không người lái (ROV - Remotely Operated Vehicle). Qua đó, chương này làm rõ mục tiêu, phạm vi nghiên cứu, phương pháp tiếp cận và định hướng công nghệ được lựa chọn để xây dựng hệ thống phần mềm quản lý nhiệm vụ. Cuối cùng, chương 1 sẽ phác thảo cấu trúc tổng thể của các chương tiếp theo trong đề án.

1.1 Đặt vấn đề

Trong những năm gần đây, sự phát triển của công nghệ robot dưới nước, đặc biệt là các phương tiện vận hành từ xa (ROV), đã mang lại những bước tiến lớn trong các lĩnh vực khảo sát địa chất, bảo trì đường ống ngầm, cứu hộ và nghiên cứu sinh vật biển. Các thiết bị ROV thường được trang bị đa dạng các loại cảm biến như cảm biến nhiệt độ, áp suất, độ sâu, con quay hồi chuyển, cùng với hệ thống camera độ nét cao để thu thập dữ liệu hình ảnh và video liên tục. Tuy nhiên, việc vận hành ROV ngoài thực địa thường diễn ra trong môi trường khắc nghiệt, không có kết nối mạng Internet ổn định, dẫn đến toàn bộ dữ liệu thu thập được phải lưu trữ cục bộ. Sau mỗi lượt lặn, người điều khiển phải đối mặt với lượng dữ liệu khổng lồ và rời rạc, bao gồm hàng loạt tệp tin phương tiện dung lượng lớn và các tệp nhật ký cảm biến thô.

Quá trình tổng hợp, đồng bộ và phân tích hậu kỳ các nguồn dữ liệu này đang tiêu tốn rất nhiều thời gian và công sức. Việc tìm kiếm mỗi tương quan giữa một thời điểm cụ thể trong đoạn video và trạng thái của các cảm biến (ví dụ: nhiệt độ và độ sâu tại vị trí xuất hiện vật thể lạ) phần lớn được thực hiện thủ công, dễ dẫn đến sai sót và bỏ lọt thông tin. Bên cạnh đó, việc nhận diện, đánh dấu các đối tượng quan trọng dưới nước hoặc lập báo cáo tóm tắt cho từng chuyến công tác đòi hỏi nhiều nhân lực chuyên môn để xem xét lại toàn bộ video. Sự thiếu hụt một hệ thống phần mềm quản lý tập trung và tự động hóa các khâu phân tích này không chỉ làm giảm hiệu suất làm việc mà còn lãng phí giá trị to lớn của nguồn dữ liệu thu thập được. Từ tình hình thực tế trên, việc phát triển một nền tảng chuyên biệt để quản lý, đồng bộ hóa và hỗ trợ phân tích thông minh dữ liệu cho ROV trở thành một nhu cầu hết sức thiết thực và cấp thiết.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay, trên thị trường và trong các nghiên cứu học thuật đã có một số phần mềm thương mại đi kèm với các dòng ROV đắt tiền, cung cấp tính năng truyền phát video trực tiếp và theo dõi thông số cảm biến. Tuy nhiên, đa số các công cụ này

thiếu tính linh hoạt trong việc chia sẻ, quản lý tập trung dựa trên nền tảng điện toán đám mây và rất hiếm khi được tích hợp sẵn các công nghệ trí tuệ nhân tạo hiện đại để hỗ trợ phân tích hậu kỳ. Hầu hết các hệ thống mã nguồn mở lại chủ yếu phục vụ cho mục đích điều khiển phần cứng cấp thấp mà bỏ ngỏ khía cạnh quản lý hệ thống thông tin nhiệm vụ.

Dựa trên những phân tích trên, đề án này đặt ra mục tiêu xây dựng một phần mềm quản lý thông tin nhiệm vụ dành riêng cho quá trình vận hành thiết bị ROV. Hệ thống hướng đến việc giải quyết trọn vẹn bài toán tải lên, lưu trữ và tự động đồng bộ hóa dữ liệu hình ảnh cùng tệp thông số cảm biến theo từng lượt lặn. Trọng tâm của đề án là cung cấp một giao diện bảng điều khiển trực quan, cho phép người dùng quan sát song song đoạn video đang phát và sự biến thiên của biểu đồ môi trường tại chính thời điểm đó. Hơn thế nữa, đề án đặt mục tiêu ứng dụng các mô hình học máy hiện đại để tự động nhận diện vật thể trong video và tự động tổng hợp báo cáo phân tích cho toàn bộ nhiệm vụ.

Về phạm vi nghiên cứu, đề án tập trung hoàn toàn vào việc phát triển nền tảng phần mềm trên nền web (Web Application) bao gồm cả ứng dụng máy chủ và giao diện người dùng. Đề án sẽ không bao gồm việc chế tạo phần cứng cho ROV hay xây dựng các giao thức truyền tải tín hiệu điều khiển thời gian thực qua sóng radio. Toàn bộ dữ liệu được sử dụng trong hệ thống giả định đã được thiết bị thu thập cục bộ tại thực địa và được người vận hành chủ động tải lên nền tảng đám mây sau khi kết thúc chuyến công tác.

1.3 Định hướng giải pháp

Để giải quyết bài toán xử lý lượng dữ liệu khổng lồ và đáp ứng các yêu cầu phân tích thông minh, đề án đề xuất xây dựng hệ thống theo kiến trúc hướng dịch vụ trên nền tảng đám mây. Trước tiên, giao diện tương tác (Frontend) được phát triển sử dụng thư viện ReactJS, giúp tạo ra các thành phần giao diện đáp ứng nhanh, đóng vai trò sống còn trong việc đồng bộ hóa thanh tiến trình phát video với trục thời gian thực của các biểu đồ đa biến. Phía máy chủ (Backend) sử dụng môi trường thực thi NodeJS kết hợp cùng hệ quản trị cơ sở dữ liệu phi cấu trúc MongoDB, mang lại khả năng lưu trữ linh hoạt cho các bản ghi dữ liệu cảm biến đa dạng. Việc lưu trữ và phân phối các tệp tin phương tiện kích thước lớn sẽ được giao phó cho nền tảng Amazon Web Services (AWS S3) thông qua cơ chế tải lên trực tiếp.

Một trong những công nghệ cốt lõi được áp dụng trong đề án là kỹ thuật thị giác máy tính dựa trên mô hình học sâu YOLOv8. Kỹ thuật này được đóng gói dưới dạng một dịch vụ vi mô (microservice) độc lập, có nhiệm vụ tự động dò tìm và

theo dõi quỹ đạo của các vật thể đáng chú ý dưới môi trường nước. Song song với đó, hệ thống ứng dụng mô hình ngôn ngữ lớn (Gemini) để đánh giá dữ liệu và tự động khởi tạo các văn bản báo cáo tóm tắt. Để đảm bảo trải nghiệm người dùng luôn mượt mà và không bị nghẽn bởi các tác vụ trí tuệ nhân tạo nặng nề, giải pháp xử lý công việc thông qua hàng đợi bất đồng bộ (Bull Queue) và truyền tải sự kiện thời gian thực (Server-Sent Events) đã được lựa chọn làm cơ chế giao tiếp chính. Sự kết hợp chặt chẽ giữa các công nghệ tiên tiến này được kỳ vọng sẽ hình thành nên một hệ sinh thái phần mềm toàn diện và thông minh cho công tác vận hành ROV.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày tổng quan về quá trình khảo sát thực trạng, phân tích chi tiết các yêu cầu chức năng và yêu cầu phi chức năng thiết yếu của hệ thống. Đồng thời, chương này cũng đưa ra các đặc tả ca sử dụng (Use Case) nền tảng làm cơ sở cho các quyết định thiết kế.

Chương 3 giới thiệu các cơ sở lý thuyết và các khung làm việc công nghệ được lựa chọn sử dụng trong dự án. Trọng tâm của chương hướng đến việc làm rõ cơ chế hoạt động của hàng đợi tác vụ bất đồng bộ, phương thức đẩy dữ liệu trực tiếp từ máy chủ đến trình duyệt và kiến trúc cấu tạo của mô hình nhận diện vật thể YOLOv8.

Trong Chương 4, đồ án trình bày chi tiết về toàn bộ quá trình phân tích và thiết kế hệ thống. Nội dung bao gồm việc phác thảo kiến trúc tổng thể, mô hình hóa cơ sở dữ liệu và thiết kế trải nghiệm người dùng đặc thù dành riêng cho việc quan sát dữ liệu đồng bộ.

Chương 5 đi sâu phân tích các giải pháp kỹ thuật cốt lõi và nhấn mạnh những đóng góp nổi bật nhất của đồ án. Các thuật toán ánh xạ thời gian giữa video và dữ liệu cảm biến, quy trình ứng dụng thị giác máy tính, cũng như phương thức tự động hóa báo cáo sẽ được minh họa một cách tường minh.

Cuối cùng, Chương 6 đóng vai trò tổng kết lại toàn bộ khối lượng công việc và những kết quả đã đạt được. Chương này cũng chỉ ra những hạn chế còn tồn đọng của hệ thống và đề xuất các hướng nghiên cứu, cải tiến cụ thể trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 2 trình bày quá trình khảo sát thực trạng công tác quản lý dữ liệu ROV hiện nay, từ đó chỉ ra những hạn chế và rút ra bài toán cần giải quyết. Dựa trên kết quả khảo sát, chương này đặc tả chi tiết các yêu cầu chức năng, yêu cầu phi chức năng của hệ thống, đồng thời cung cấp biểu đồ Use Case và quy trình nghiệp vụ tổng quát nhằm định hình kiến trúc thiết kế trong các chương tiếp theo.

2.1 Khảo sát hiện trạng

Trong thực tế vận hành ROV, quá trình thu thập và phân tích dữ liệu sau chuyến đi đang tồn tại nhiều bất cập. Phần cứng trên ROV ghi lại dữ liệu cảm biến (nhiệt độ, áp suất, độ sâu, GPS) dưới định dạng CSV hoặc JSON, đồng thời lưu video vào ổ cứng của thiết bị điều khiển. Sau mỗi lượt lặn, người điều khiển (Operator) phải chép thủ công các tệp về máy tính cá nhân. Đặc biệt, do môi trường thực địa không có kết nối mạng ổn định, toàn bộ quá trình upload diễn ra sau khi đoàn về bờ.

Công tác phân tích hậu kỳ hiện tại diễn ra hoàn toàn rời rạc: người phân tích phải mở trình phát video, sau đó đối chiếu thủ công từng dòng trong tệp CSV để xác định trạng thái môi trường tại một khoảnh khắc cụ thể trong video. Quy trình này không chỉ mất nhiều thời gian, dễ sai sót, mà còn gây khó khăn lớn trong việc chia sẻ kết quả và cộng tác giữa các thành viên trong đội.

Khảo sát các phần mềm hiện có cho thấy chưa có giải pháp nào đáp ứng đầy đủ nhu cầu quản lý dữ liệu hậu kỳ cho ROV trên nền tảng web đa người dùng. Bảng 2.1 tóm tắt sự so sánh giữa các giải pháp phổ biến với hệ thống đề xuất.

Bảng 2.1: So sánh các giải pháp quản lý dữ liệu ROV hiện có

Tiêu chí	QGround-Control	BlueOS	Google Drive	Đề xuất
Kiểm soát ROV thời gian thực	✓	✓	—	—
Phân tích dữ liệu hậu kỳ	Hạn chế	—	—	✓
Đồng bộ Video – Cảm biến	—	—	—	✓
Nhận diện vật thể AI (YOLO)	—	—	—	✓
Tóm tắt nhiệm vụ bằng LLM	—	—	—	✓
Nền tảng Web đa người dùng	—	Hạn chế	✓	✓
Phân quyền theo vai trò (RBAC)	—	—	—	✓

Từ phân tích trên, hệ thống “*Quản lý Thông tin Nhiệm vụ ROV*” được đề xuất với bốn tính năng cốt lõi:

1. Quản lý tập trung các Chuyến khảo sát (Projects) và Lượt lặn (Trips) theo mô

hình phân cấp.

2. Ảnh xạ và đồng bộ thanh thời gian video với biểu đồ dữ liệu cảm biến theo thời gian thực.
3. Tích hợp học sâu (YOLOv8) để tự động nhận diện vật thể và trích xuất bằng chứng từ video.
4. Ứng dụng LLM (Gemini) để tổng hợp báo cáo nhiệm vụ bằng ngôn ngữ tự nhiên.

2.2 Tổng quan chức năng

Hệ thống phục vụ ba nhóm người dùng chính, với quyền hạn được phân chia theo cơ chế kiểm soát truy cập dựa trên vai trò (RBAC):

- **Viewer (Người xem):** Được truy cập hệ thống để xem danh sách chuyến đi, theo dõi video, xem biểu đồ cảm biến và các báo cáo AI. Đây là vai trò mặc định khi một tài khoản mới được tạo.
- **Operator (Người vận hành):** Kế thừa toàn bộ quyền của Viewer, đồng thời có thể tạo và chỉnh sửa Project/Trip/ROV; tải lên và xóa dữ liệu thực địa (CSV/DVL/Sonar); tải lên tệp phương tiện; tạo và xóa bằng chứng (Evidence); kích hoạt phân tích YOLO và tạo tóm tắt AI.
- **Admin (Quản trị viên):** Kế thừa toàn bộ quyền của Operator, bổ sung đặc quyền xóa Project/Trip/ROV/Media, quản lý và phân quyền người dùng, xem nhật ký hoạt động (Audit Log).

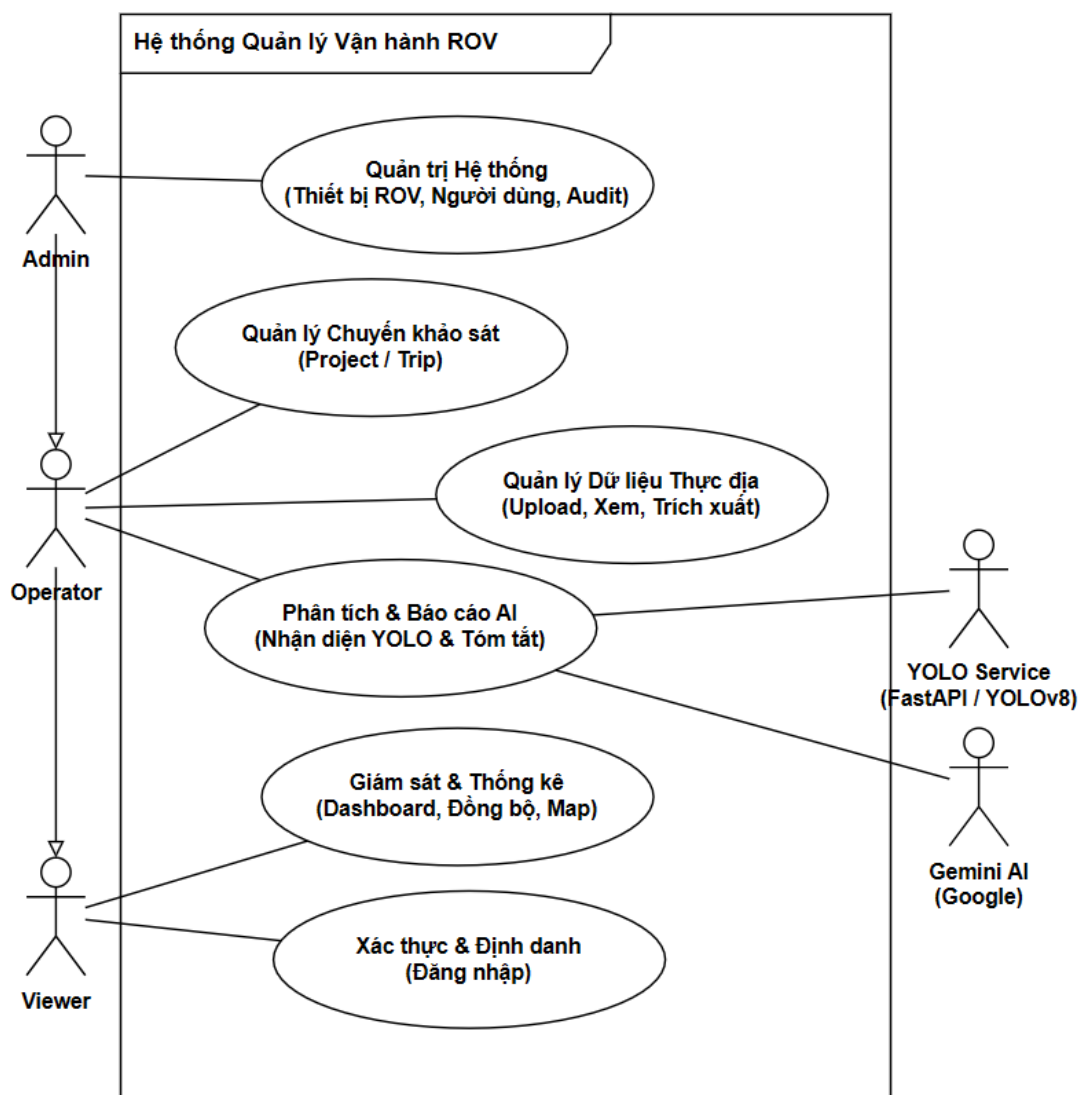
Ma trận phân quyền chi tiết được trình bày trong Bảng 2.2.

Bảng 2.2: Ma trận phân quyền RBAC

Tính năng	Admin	Operator	Viewer
Xem Dashboard, ROV, Project, Trip	✓	✓	✓
Nhận thông báo realtime (SSE)	✓	✓	✓
Tạo và chỉnh sửa Project / Trip	✓	✓	—
Tạo và chỉnh sửa Thiết bị ROV	✓	✓	—
Tải lên dữ liệu cảm biến (CSV / DVL / Sonar)	✓	✓	—
Xóa dữ liệu cảm biến / DVL / Sonar	✓	✓	—
Tải lên Media (Video / Ảnh)	✓	✓	—
Trích xuất Bằng chứng (Photo / Clip)	✓	✓	—
Xóa Bằng chứng (Photo / Clip)	✓	✓	—
Kích hoạt phân tích YOLO	✓	✓	—
Tạo báo cáo tóm tắt AI	✓	✓	—
Xuất dữ liệu (CSV / PDF)	✓	✓	—
Xóa Media (Ảnh / Video)	✓	—	—
Xóa Project / Trip	✓	—	—
Xóa Thiết bị ROV	✓	—	—
Quản lý và phân quyền Người dùng	✓	—	—
Xem Nhật ký hoạt động (Audit Log)	✓	—	—

2.2.1 Biểu đồ use case tổng quát

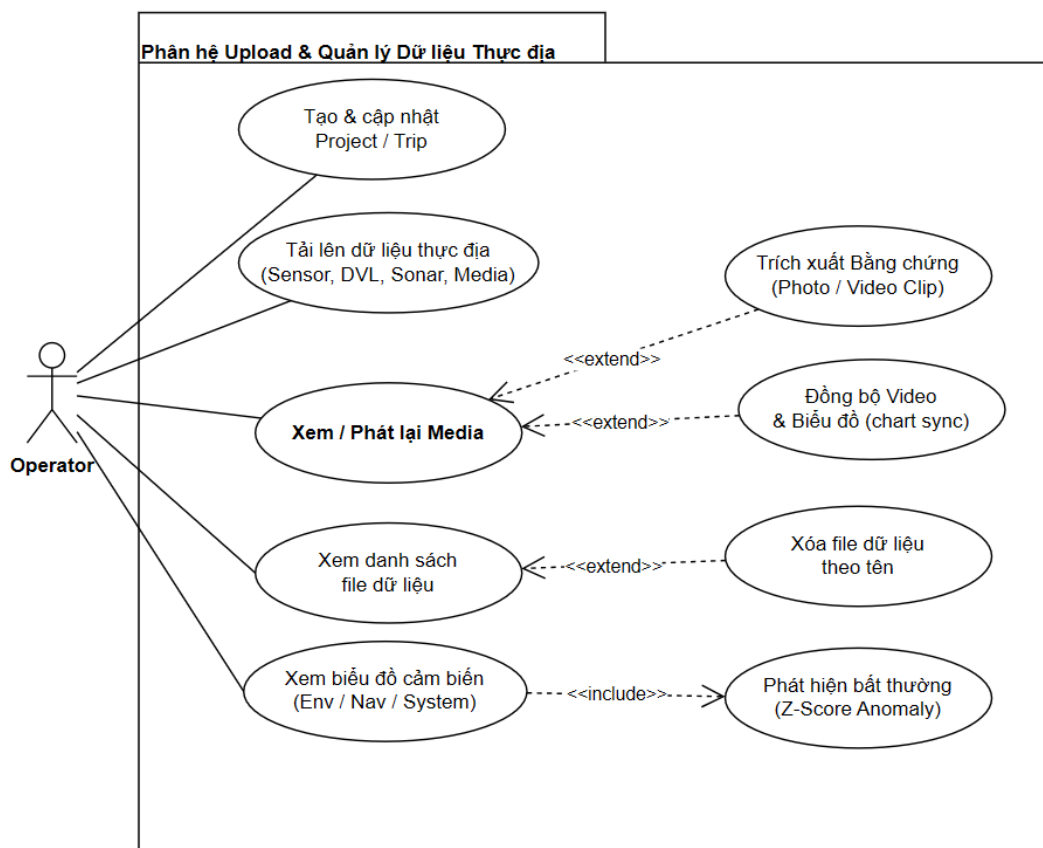
Biểu đồ Use Case tổng quát (Hình 2.1) mô tả bức tranh toàn cảnh về sự tương tác giữa các tác nhân và hệ thống. Hệ thống có năm tác nhân: ba tác nhân người dùng (Viewer, Operator, Admin) kế thừa quyền theo chiều tăng dần, và hai tác nhân hệ thống bên ngoài là YOLO Service (FastAPI/YOLOv8) và Gemini AI (Google) tham gia vào các tác vụ phân tích. Các nhóm chức năng chính bao gồm: Quản lý danh mục ROV, Quản lý chuyến khảo sát và lượt lặn, Quản lý dữ liệu thực địa và phương tiện, và Tương tác với trí tuệ nhân tạo.



Hình 2.1: Biểu đồ Use Case tổng quát của hệ thống

2.2.2 Biểu đồ use case phân rã Phân hệ Dữ liệu Thực địa

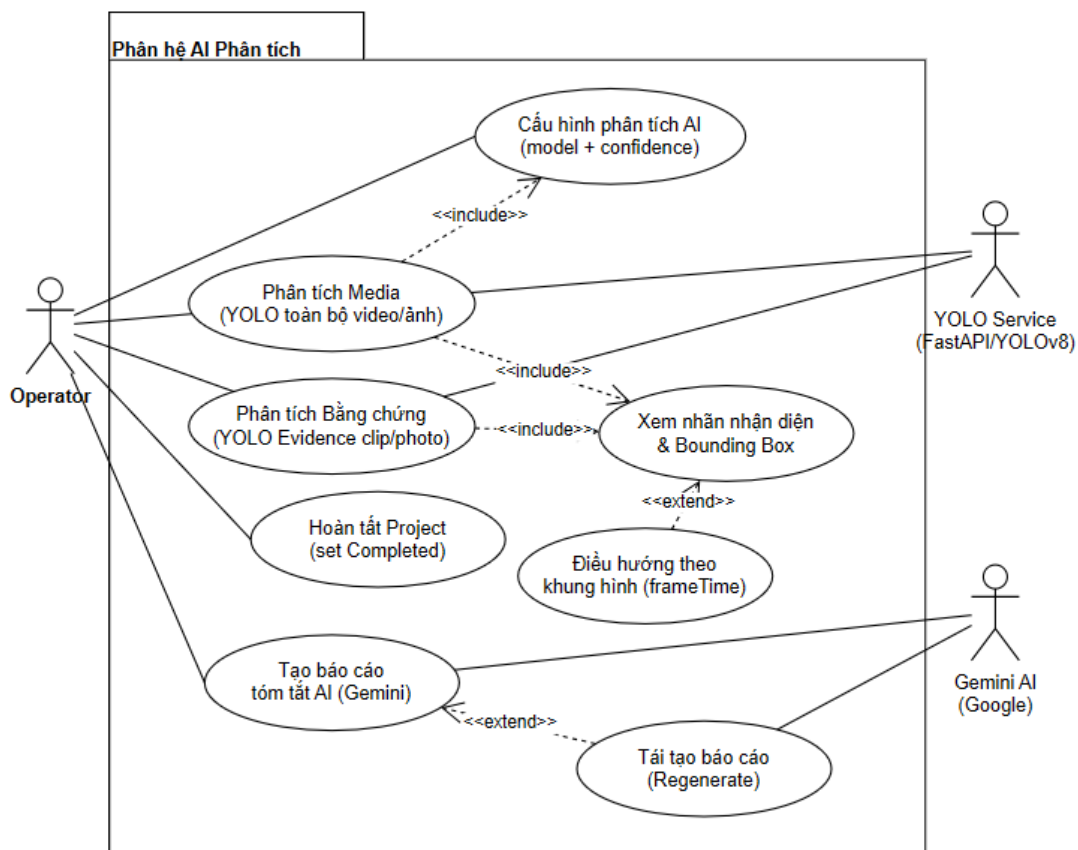
Phân hệ upload và quản lý dữ liệu thực địa là lõi nghiệp vụ của hệ thống. Hình 2.2 làm rõ các thao tác chi tiết của Operator sau khi một lượt lặn được khởi tạo. Operator tải lên nhiều loại tệp dữ liệu: CSV cảm biến (hỗ trợ nhiều file, tự động phát hiện chồng lấp thời gian), JSON DVL và tệp Sonar. Hệ thống lưu từng tệp theo tên nguồn (*sourceFile*), cho phép xóa từng file riêng lẻ mà không ảnh hưởng đến các file còn lại. Nếu video có trường *recordedAt* (thời điểm bắt đầu quay), chế độ đồng bộ Video–Biểu đồ được kích hoạt tự động. Trong khi xem, Operator có thể trích xuất bằng chứng (Photo/Clip) từ video đang đồng bộ.



Hình 2.2: Biểu đồ Use Case phân rã Phân hệ Upload và Quản lý Dữ liệu Thực địa

2.2.3 Biểu đồ use case phân rã Phân hệ AI Phân tích

Phân hệ AI Phân tích là điểm khác biệt lớn nhất của hệ thống so với các giải pháp truyền thống. Hình 2.3 thể hiện sự tham gia của hai tác nhân hệ thống bên ngoài: YOLO Service (Python FastAPI) và Gemini AI (Google). Operator cấu hình tham số trước khi chạy phân tích (lựa chọn mô hình và ngưỡng confidence), sau đó kích hoạt phân tích cho toàn bộ tệp phương tiện hoặc riêng từng bằng chứng đã trích xuất. Kết quả nhận diện hiển thị dưới dạng bounding box đồng bộ theo từng khung hình; Operator có thể nhấp vào nhãn để lọc và chỉ hiển thị bounding box của lớp đó. Khi Project hoàn tất, Operator kích hoạt Gemini AI để sinh báo cáo tóm tắt bằng ngôn ngữ tự nhiên và có thể tái tạo nếu cần.



Hình 2.3: Biểu đồ Use Case phân rã Phân hệ AI Phân tích

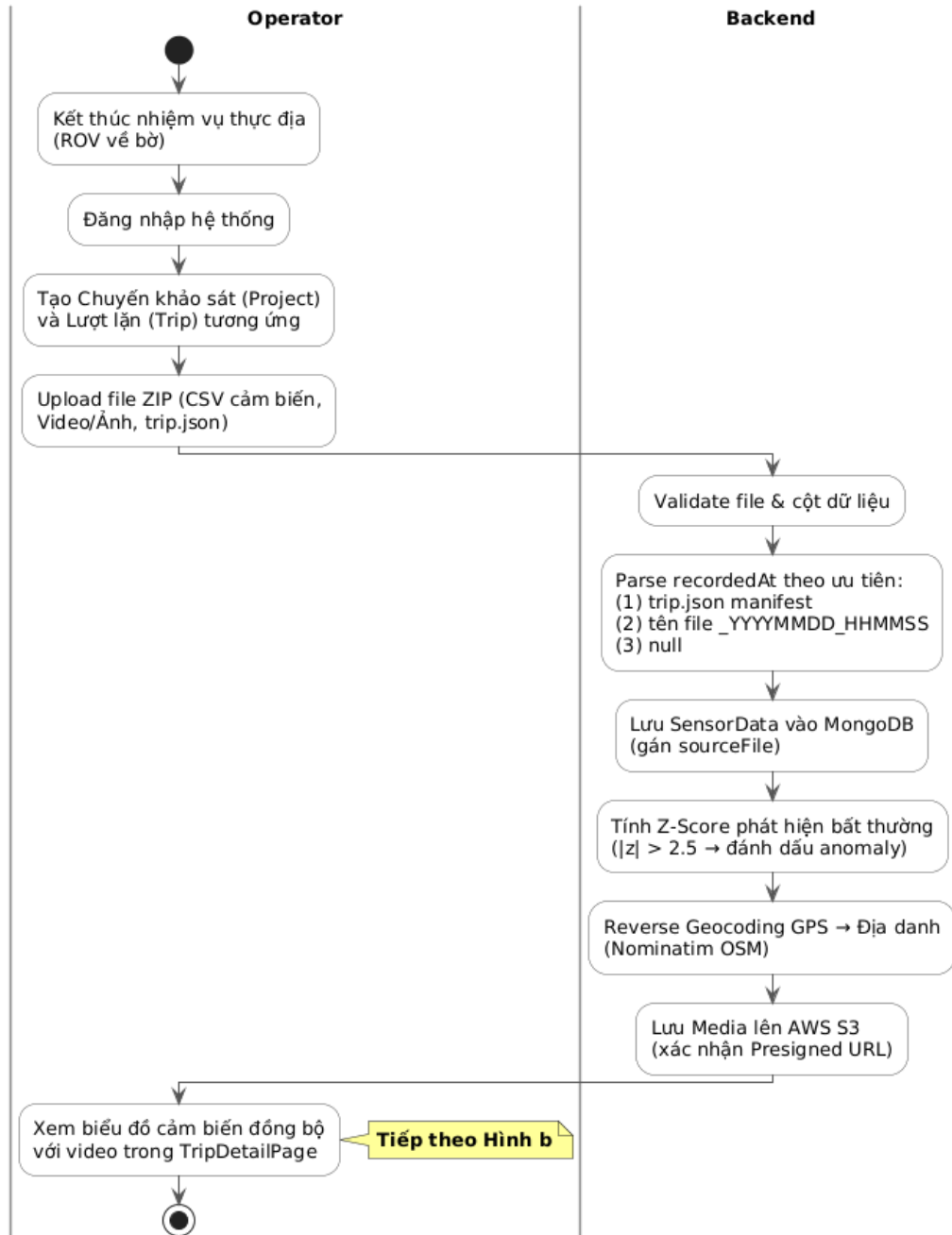
2.2.4 Quy trình nghiệp vụ tải lên và phân tích dữ liệu

Hình ?? mô tả luồng nghiệp vụ đầu cuối từ khi Operator trở về từ thực địa cho đến khi có báo cáo tổng kết, bao gồm năm giai đoạn chính:

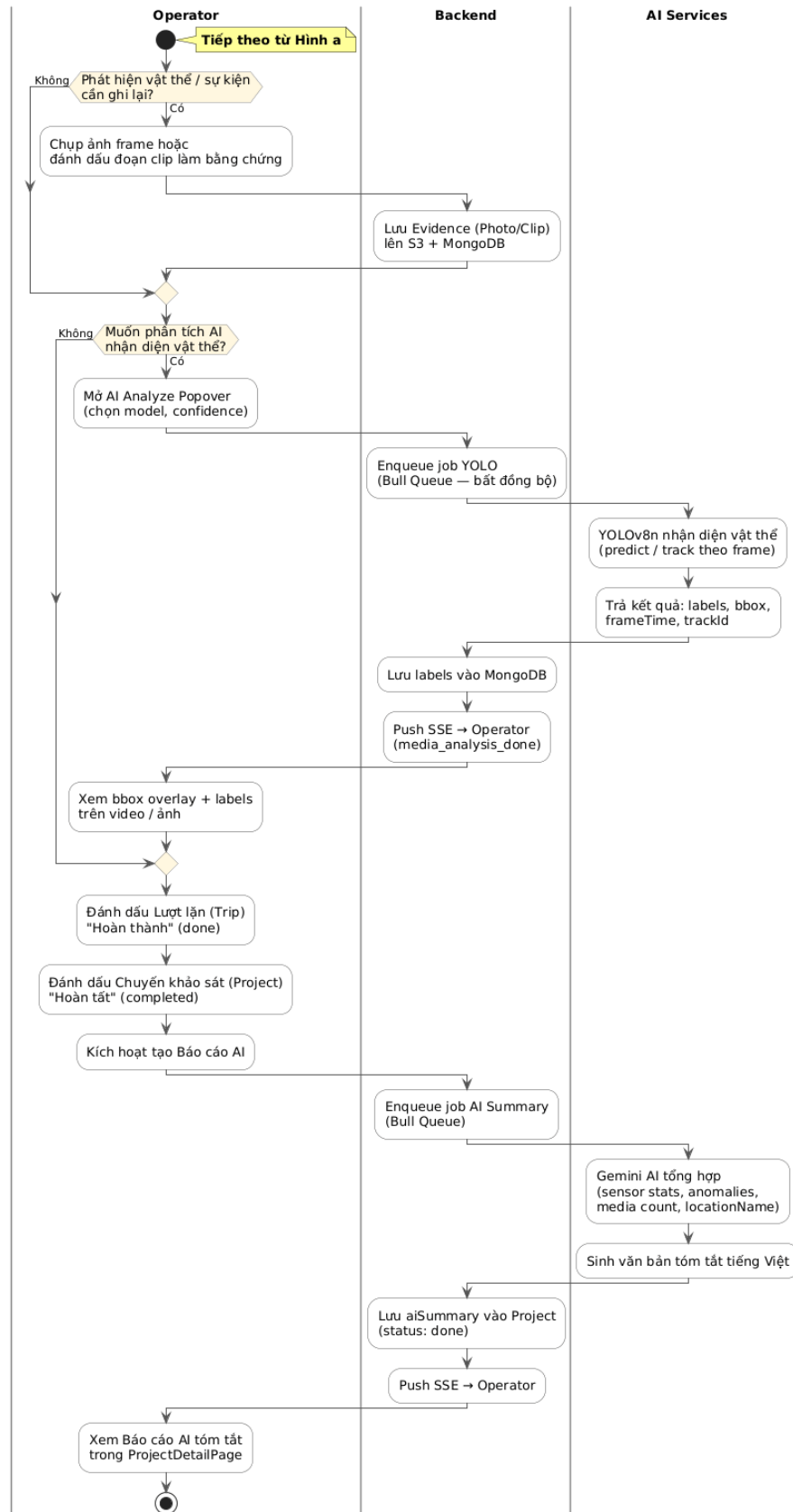
1. Đăng nhập và tạo mới Chuyến khảo sát (Project) cùng các Lượt lặn (Trips).
2. Tải lên toàn bộ tệp phương tiện và tệp dữ liệu cảm biến tương ứng với từng lượt lặn.
3. Xem lại video, trích xuất bằng chứng và kích hoạt phân tích nhận diện vật thể (YOLOv8).
4. Đánh dấu hoàn tất chuyến đi (trạng thái *Completed*).
5. Kích hoạt AI tóm tắt (Gemini) để sinh ra văn bản báo cáo nhiệm vụ.

2.3 Đặc tả chức năng

Phần này đặc tả chi tiết bốn Use Case cốt lõi, thể hiện tính đột phá của hệ thống so với các giải pháp hiện có. Các Use Case liên quan đến quản lý người dùng và CRUD cơ bản (Project, Trip, ROV) được đặc tả bổ sung trong Phụ lục B.



Hình 2.4: Biểu đồ hoạt động quy trình phân tích dữ liệu nhiệm vụ — Giai đoạn a: Thu thập và xử lý dữ liệu



Hình 2.5: Biểu đồ hoạt động quy trình phân tích dữ liệu nhiệm vụ — Giai đoạn b: Phân tích và tổng kết

2.3.1 UC-01: Tải lên và đồng bộ dữ liệu cảm biến

Bảng 2.3: Đặc tả Use Case UC-01: Tải lên và đồng bộ dữ liệu cảm biến

Mã Use Case	UC-01
Tên	Tải lên và đồng bộ dữ liệu cảm biến (Sensor Data Sync)
Tác nhân	Operator
Tiền điều kiện	Tác nhân đã đăng nhập và đang ở trang chi tiết Lượt lặn (TripDetailPage).
Luồng chính	(1) Tác nhân kéo thả hoặc chọn tệp CSV/DVL/Sonar vào khu vực tải lên. (2) Hệ thống xác thực định dạng tệp (CSV phải chứa hai cột bắt buộc: timestamp và depth; các cột còn lại như temp, pressure, yaw, voltage,... là tùy chọn). (3) Hệ thống bóc tách tọa độ GPS từ dòng đầu tiên, gọi API Nominatim để chuyển thành tên địa danh (<i>Reverse Geocoding</i>). (4) Hệ thống lưu dữ liệu vào MongoDB, tính toán phát hiện bất thường bằng Z-Score ($z > 2,5$). (5) Giao diện hiển thị biểu đồ môi trường; khi video phát, thanh đánh dấu dọc di chuyển đồng bộ theo dấu thời gian.
Luồng phát sinh	Định dạng tệp không hợp lệ → hệ thống thông báo lỗi chi tiết và yêu cầu tải lại. Tệp trùng phạm vi thời gian → hệ thống cảnh báo số dòng bị bỏ qua (overlap warning).
Hậu điều kiện	Dữ liệu cảm biến được liên kết thành công vào lượt lặn; biểu đồ và bản đồ GPS cập nhật tức thì.

2.3.2 UC-02: Kích hoạt phân tích nhận diện vật thể bằng YOLOv8

Bảng 2.4: Đặc tả Use Case UC-02: Phân tích nhận diện vật thể bằng AI

Mã Use Case	UC-02
Tên	Kích hoạt phân tích nhận diện vật thể (AI Object Detection)
Tác nhân	Operator; YOLO Service (hệ thống bên ngoài)
Tiền điều kiện	Tệp video hoặc ảnh đã được tải lên thành công (trạng thái <i>Ready</i>).

Luồng chính	(1) Tác nhân nhấn nút Cài đặt AI trên thanh công cụ video; cửa sổ thiết lập hiện ra cho phép chọn mô hình (YOLOv8n/tùy chỉnh) và ngưỡng độ tin cậy. (2) Tác nhân nhấn “<i>Chạy phân tích</i>”; hệ thống đưa tác vụ vào hàng đợi Bull Queue và trả về xác nhận tiếp nhận (HTTP 202). (3) Worker nền gọi Python Microservice, truyền URL presigned của tệp phương tiện, mô hình và ngưỡng. (4) YOLO Service xử lý từng khung hình (video: lấy mẫu theo tần suất thích nghi), trả về danh sách nhãn kèm tọa độ bounding box và thời điểm khung hình (frameTime). (5) Hệ thống lưu kết quả vào MongoDB, đẩy thông báo SSE đến giao diện tác nhân. (6) Bounding box xuất hiện trên video đồng bộ theo từng khung hình đang phát.
Luồng phát sinh	YOLO Service không phản hồi hoặc quá tải → job thất bại sau 1 lần thử (timeout 25 phút); Worker ghi <code>analysisStatus = 'failed'</code> và thông báo lỗi qua SSE.
Hậu điều kiện	Nhãn nhận diện (Labels, Bbox, FrameTime, TrackId) được lưu và hiển thị theo thời gian thực trên giao diện.

2.3.3 UC-03: Trích xuất và phân tích bằng chứng

Bảng 2.5: Đặc tả Use Case UC-03: Trích xuất bằng chứng (Evidence System)

Mã Use Case	UC-03
Tên	Trích xuất và phân tích bằng chứng (Evidence System)
Tác nhân	Operator
Tiền điều kiện	Video của lượt lặn đang được phát trên TripDetailPage.

Luồng chính	(1) Tác nhân phát hiện vật thể nghi vấn, nhấn <i>Photo</i> hoặc <i>Clip</i> trên thanh công cụ. (2a) <i>Photo</i>: Hệ thống trích xuất khung hình hiện tại thành tệp PNG, tải lên AWS S3. Nếu bounding box đang hiển thị, tọa độ được ghi vào ảnh (canvas burn-in). (2b) <i>Clip</i>: Tác nhân nhấn bắt đầu → dừng; hệ thống ghi nhận t_{start} và t_{end} mà không cắt tệp vật lý. (3) Bằng chứng được lưu vào collection <i>Snapshots</i>; tác nhân có thể ghi chú thêm. (4) Tác nhân tùy chọn kích hoạt YOLO riêng cho bằng chứng này (mở rộng từ UC-02).
Luồng phát sinh	Không có video đang active → nút bị vô hiệu hóa, không thể thao tác.
Hậu điều kiện	Bằng chứng được lưu thành công và hiển thị trong bảng Evidence; tệp video gốc không bị thay đổi.

2.3.4 UC-04: Tạo báo cáo tóm tắt bằng ngôn ngữ tự nhiên

Bảng 2.6: Đặc tả Use Case UC-04: Tóm tắt nhiệm vụ bằng AI (AI Summary)

Mã Use Case	UC-04
Tên	Tạo báo cáo tóm tắt nhiệm vụ bằng AI (AI Summary)
Tác nhân	Operator; Gemini AI (hệ thống bên ngoài)
Tiền điều kiện	Trạng thái Chuyển khảo sát (Project) đã chuyển sang <i>Completed</i> .
Luồng chính	(1) Tác nhân truy cập trang ProjectDetailPage, nhấn nút “<i>Tạo Tóm tắt</i>”. (2) Hệ thống thu thập thông kê chuyển đi: địa điểm (locationName), thời gian, số lượt lặn, tổng số ảnh/video, số cảnh báo cảm biến. (3) Dữ liệu được đưa vào mẫu câu (Prompt) và gửi tới Gemini API qua Bull Queue. (4) Hệ thống nhận phản hồi, lưu văn bản tóm tắt vào trường project.aiSummary. (5) Thông báo SSE được đẩy đến người kích hoạt; tóm tắt hiện ra tức thì trên giao diện.

Luồng phát sinh	Gemini API không phản hồi → job thất bại, trạng thái chuyển về <i>failed</i> ; tác nhân có thể thử lại.
Hậu điều kiện	Vấn bản tóm tắt xuất hiện trên ProjectDetailPage, giúp ban quản lý nắm bắt nhanh kết quả chuyển đi.

2.4 Yêu cầu phi chức năng

Để đảm bảo hệ thống hoạt động ổn định và mang lại trải nghiệm người dùng tối ưu trong môi trường vận hành thực tế, phần mềm cần thỏa mãn các yêu cầu phi chức năng sau:

- **Hiệu năng và phản hồi (Performance):** Giao diện cockpit theo dõi video và cảm biến không được giật lag. Cơ chế đồng bộ thanh thời gian video–biểu đồ phải duy trì độ trễ dưới 100ms. Thông báo SSE phải hiển thị tức thì khi có thay đổi trạng thái.
- **Xử lý bất đồng bộ (Asynchronous Processing):** Các tác vụ nặng như phân tích hàng nghìn khung hình video bằng YOLOv8 hoặc truy vấn LLM (Gemini) không được chặn (block) luồng giao diện người dùng. Tất cả tác vụ này phải được xếp vào hàng đợi (Bull Queue) và xử lý ngầm, kết quả trả về qua SSE.
- **Khả năng lưu trữ (Scalability & Storage):** Tập dữ liệu thực địa (video, ảnh, sonar) thường có kích thước lớn (Gigabytes). Hệ thống không lưu tệp trực tiếp trong cơ sở dữ liệu mà sử dụng nền tảng lưu trữ đám mây (AWS S3) với cơ chế Presigned URL, giảm tải băng thông cho máy chủ chính.
- **Bảo mật và phân quyền (Security):** Hệ thống sử dụng JWT có vòng đời ngắn (15 phút) kết hợp Refresh Token (7 ngày) và danh sách đen (Blacklist) trên Redis để vô hiệu hóa tức thì tài khoản đăng xuất hoặc bị chặn. Dữ liệu phân lập nghiêm ngặt theo vai trò; mọi truy cập tài nguyên đều kiểm tra quyền tại lớp middleware.
- **Khả năng sử dụng (Usability):** Giao diện hỗ trợ cả chế độ sáng (Light) và tối (Dark), tương thích trên màn hình từ 375px (mobile) đến 1440px (desktop). Tất cả danh sách có trạng thái trống (empty state) thân thiện; các thao tác tải lên có phản hồi tiến trình rõ ràng.

2.5 Sơ đồ trạng thái

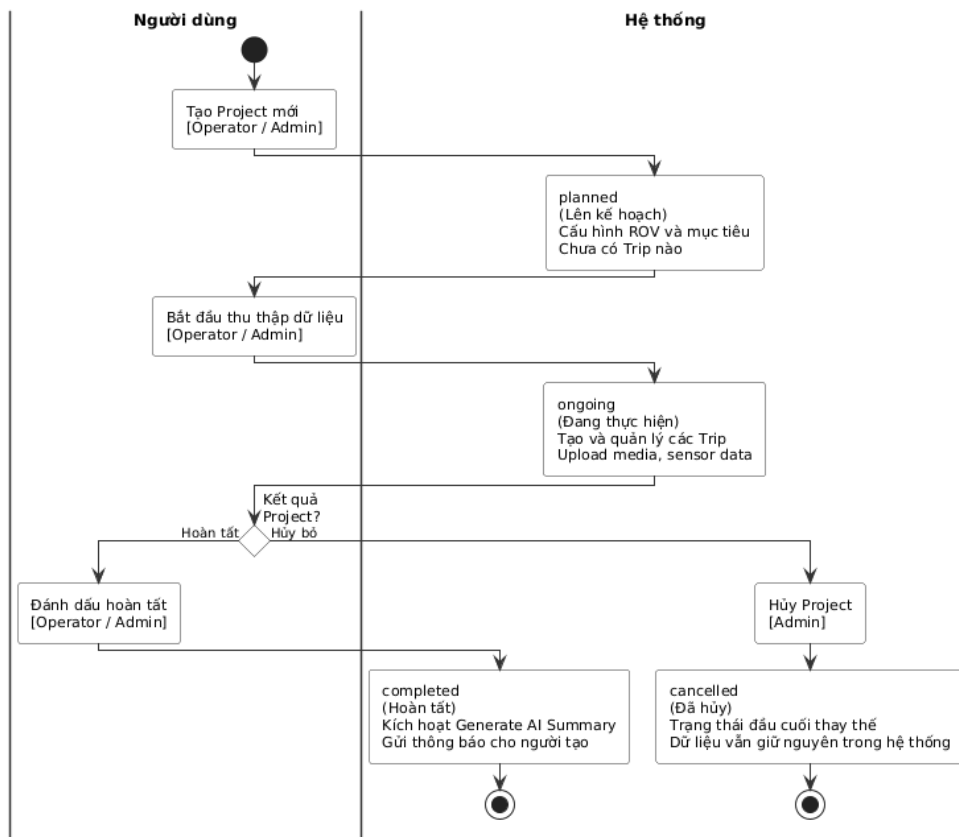
Hệ thống quản lý ROV có hai thực thể trung tâm với vòng đời nhiều giai đoạn: **Chuyến khảo sát (Project)** và **Lượt lặn (Trip)**. Ngoài ra, mỗi tệp phương tiện (Media) có vòng đời riêng gắn với quá trình phân tích AI. Việc mô hình hóa chính xác các trạng thái và điều kiện chuyển đổi là nền tảng để thiết kế backend không

mắc lỗi logic nghiệp vụ.

2.5.1 Vòng đời Chuyển khảo sát (Project)

Hình 2.6 mô tả sơ đồ hoạt động của một Chuyển khảo sát với bốn trạng thái: *planned*, *ongoing*, *completed* và *cancelled*. Trong hệ thống, status của Project đóng vai trò nhân theo dõi tiến độ — không có ràng buộc nào tại lớp backend ngăn cản việc tải lên dữ liệu hay chỉnh sửa nội dung khi Project ở bất kỳ trạng thái nào. Hành vi duy nhất phụ thuộc vào status là: chỉ Project ở trạng thái *completed* mới được phép tạo báo cáo tóm tắt AI.

Sau khi được tạo mới, Project bắt đầu ở trạng thái *planned* — giai đoạn lên kế hoạch trước khi thực địa bắt đầu. Khi Operator hoặc Admin kích hoạt thu thập dữ liệu, Project chuyển sang *ongoing*; đây là trạng thái chính trong suốt quá trình thực địa. Khi toàn bộ lượt lặn hoàn tất, Project được đánh dấu *completed* — trạng thái này mở khóa tính năng tạo báo cáo tóm tắt bằng Gemini AI và đẩy thông báo SSE đến người tạo Project. Nếu chiến dịch bị hủy bỏ, Admin có thể chuyển Project sang *cancelled* — đây là trạng thái đầu cuối thay thế, toàn bộ dữ liệu đã thu thập vẫn được giữ nguyên trong hệ thống.

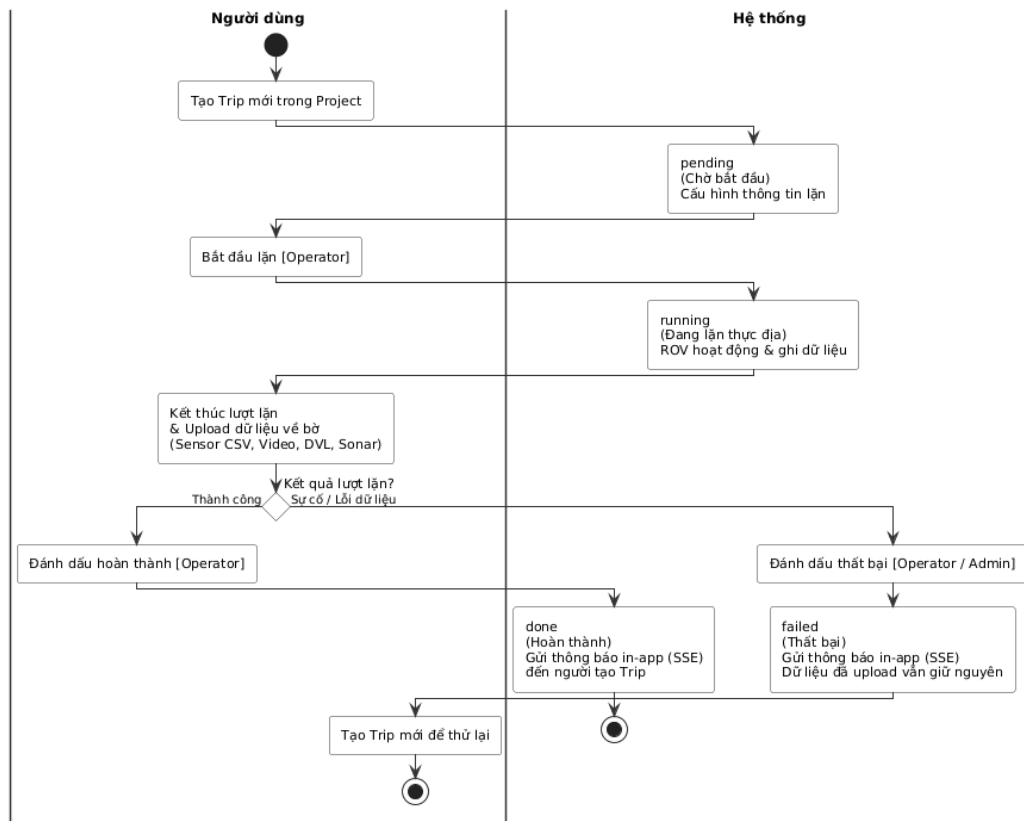


Hình 2.6: Sơ đồ trạng thái vòng đời Chuyển khảo sát (Project)

2.5.2 Vòng đời Lướt lặn (Trip)

Hình 2.7 mô tả sơ đồ hoạt động của một Lướt lặn với bốn trạng thái: *pending*, *running*, *done* và *failed*. Tương tự Project, status của Trip đóng vai trò nhãn theo dõi tiến độ — backend không áp đặt ràng buộc upload hay chỉnh sửa theo trạng thái cụ thể; các tính năng phân tích YOLO và đồng bộ video–biểu đồ được Operator kích hoạt thủ công qua giao diện, không tự động theo status.

Trip mới được tạo ở trạng thái *pending* — chứa thông tin cấu hình nhưng chưa có dữ liệu thực địa. Khi Operator bắt đầu lặn, Trip chuyển sang *running*. Sau khi kết thúc lặn và tải dữ liệu về bờ (CSV cảm biến, video, DVL, Sonar), Operator đánh dấu Trip là *done* nếu thành công — hệ thống gửi thông báo in-app (SSE) đến người tạo Trip. Nếu xảy ra sự cố, Trip chuyển sang *failed* — thông báo in-app tương tự được đẩy đến người tạo và toàn bộ dữ liệu đã tải lên vẫn được giữ nguyên; Operator có thể tạo Lướt lặn mới để thử lại.



Hình 2.7: Sơ đồ trạng thái vòng đời Lướt lặn (Trip)

2.5.3 Vòng đời phân tích Media

Mỗi tệp phương tiện (video hoặc ảnh) trong hệ thống có trường `analysisStatus` riêng, độc lập với trạng thái của Lướt lặn. Vòng đời này phản ánh tiến trình của tác vụ phân tích YOLO bất đồng bộ, được tóm tắt trong Bảng 2.7.

Bảng 2.7: Các trạng thái vòng đời phân tích Media

Trạng thái	Ý nghĩa	Điều kiện chuyển đổi
idle	Chưa phân tích	Trạng thái mặc định sau khi tệp được xác nhận tải lên thành công (<i>status = ready</i>)
pending	Đang chờ trong hàng đợi	Operator nhấn “ <i>Chạy phân tích</i> ” → job được thêm vào Bull Queue, labels cũ bị xóa
done	Phân tích hoàn tất	Worker nhận kết quả từ YOLO Service, lưu labels vào MongoDB, đẩy thông báo SSE
failed	Phân tích thất bại	YOLO Service timeout (25 phút) hoặc lỗi kết nối; Worker ghi trạng thái failed, Operator có thể kích hoạt lại bất kỳ lúc nào

Thiết kế tách biệt `analysisStatus` khỏi `status` của tệp (pending/ready/failed) cho phép tệp phương tiện có thể được xem và sử dụng bình thường trong khi quá trình phân tích AI vẫn đang diễn ra ngầm. Đây là nguyên tắc quan trọng đảm bảo luồng công việc của Operator không bị gián đoạn.

Chương 2 đã khảo sát toàn diện bối cảnh bài toán, xác định rõ chân dung người dùng, ma trận phân quyền, các luồng nghiệp vụ chính và sơ đồ trạng thái của các thực thể trung tâm. Những yêu cầu chức năng và phi chức năng được đặc tả ở trên là cơ sở để lựa chọn kiến trúc phần mềm và hệ sinh thái công nghệ, sẽ được trình bày chi tiết trong Chương 3 tiếp theo.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 3 phân tích các công nghệ cốt lõi được lựa chọn để xây dựng hệ thống, giải thích lý do ưu tiên từng giải pháp thay vì các lựa chọn thay thế, và mô tả chi tiết cơ chế hoạt động của ba thành phần kỹ thuật then chốt theo yêu cầu bố cục đồ án: hàng đợi tác vụ bất đồng bộ (Bull Queue), giao tiếp thời gian thực từ máy chủ đến trình duyệt (SSE), và kiến trúc nhận diện vật thể YOLOv8. Bảng 3.1 tóm tắt toàn bộ hệ sinh thái công nghệ.

Bảng 3.1: Hệ sinh thái công nghệ của hệ thống

Tầng / Vai trò	Công nghệ (Phiên bản)	Ghi chú
Giao diện người dùng	React 18.3, Vite 5.3, Tailwind CSS 3.4	SPA, utility-first CSS, dark mode
Quản lý trạng thái	Zustand 4.5, React Query 5.4	Client state + server state
Biểu đồ	Recharts 3.8	AreaChart, LineChart, Brush
Bản đồ	Leaflet.js 1.9 (raw API)	Không dùng react-leaflet
Máy chủ	Node.js 20 LTS, Express.js 4.19	REST API, SSE stream
ORM	Mongoose 8.4	Schema, validation, index
Cơ sở dữ liệu	MongoDB Atlas	NoSQL document, 512MB free
Cache & hàng đợi	Redis + Bull 4.16 (4 queues)	JWT blacklist + async jobs
Lưu trữ tệp	AWS S3 + Presigned URL	Upload thẳng từ trình duyệt
Xác thực	JWT (HS256) + Passport.js	Access 15 phút, Refresh 7 ngày
Giao tiếp realtime	Server-Sent Events (SSE)	Một chiều: server → client
Nhận diện vật thể	YOLOv8n + FastAPI, Python 3.11	Microservice độc lập
LLM tóm tắt	Gemini 2.5 Flash	@google/generative-ai 0.24
Phát hiện bất thường	Z-Score (tự cài đặt)	Ngưỡng $ z > 2,5$
Reverse Geocoding	OpenStreetMap Nominatim	Miễn phí, không cần API key

3.1 Nền tảng Giao diện Người dùng (Frontend)

Yêu cầu phi chức năng ở Mục 2.4 đặt ra ngưỡng khắt khe: thanh đánh dấu đồng bộ giữa video và biểu đồ cảm biến phải phản hồi dưới 100ms, giao diện phải hỗ trợ dark mode và tương thích từ màn hình 375px đến 1440px. Tập hợp thư viện sau được lựa chọn để đáp ứng những ràng buộc này.

3.1.1 React 18 và Vite

React 18.3 reactjs được lựa chọn làm nền tảng xây dựng giao diện nhờ cơ chế Virtual DOM và kiến trúc component hóa. Thay vì cập nhật trực tiếp DOM thật

mỗi khi dữ liệu cảm biến thay đổi, React duy trì một bản sao DOM ảo trong bộ nhớ, tính toán phần khác biệt (diff), rồi chỉ áp dụng tối thiểu lên DOM thật. Điều này đặc biệt quan trọng trong TripDetailPage: khi video phát, sự kiện `timeupdate` của HTML5 Video API cập nhật liên tục, React chỉ re-render đúng thành phần `ReferenceLine` trên biểu đồ thay vì vẽ lại toàn bộ trang. So với Vue.js hay Angular, React cung cấp hệ sinh thái thư viện phong phú hơn và kiểm soát tường minh hơn khi làm việc với Canvas API và các API trình duyệt cấp thấp — yêu cầu thiết yếu để xây dựng tính năng chụp ảnh frame (canvas burn-in) và bản đồ GPS.

Vite 5.3 vite thay thế Create React App làm công cụ build và máy chủ phát triển. Vite sử dụng ES Module native của trình duyệt trong môi trường development, loại bỏ bước bundle toàn bộ mã nguồn trước khi khởi động. Kết quả là thời gian khởi động máy chủ dev giảm xuống dưới 1 giây và Hot Module Replacement (HMR) phản hồi tức thì khi sửa code, tăng đáng kể tốc độ phát triển.

Tailwind CSS 3.4 tailwindcss cung cấp hệ thống lớp tiện ích (utility-first) tích hợp với thiết kế token ngữ nghĩa. Toàn bộ màu sắc trong hệ thống được định nghĩa qua biến CSS (`-background`, `-foreground`, `-card`, `-border`,...) trong `index.css` và Tailwind ánh xạ các lớp như `bg-card`, `text-foreground` sang biến CSS tương ứng. Khi người dùng chuyển dark mode, script chỉ cần thêm class `dark` vào phần tử `<html>` — toàn bộ hàng trăm component tự động đổi màu mà không cần viết CSS riêng.

3.1.2 Quản lý trạng thái: Zustand và React Query

Hệ thống phân chia trách nhiệm quản lý trạng thái thành hai lớp độc lập:

Zustand 4.5 zustand quản lý trạng thái phía client: thông tin xác thực người dùng (user object, access token, refresh token) và chế độ giao diện sáng/tối. Zustand dùng closure đơn giản thay vì Redux Store + Action + Reducer, giảm đáng kể boilerplate. Trạng thái auth được persist vào `localStorage` qua middleware `persist`, đảm bảo người dùng không bị đăng xuất khi reload trang. Khi Axios interceptor nhận lỗi 401 (access token hết hạn), nó gọi `POST /auth/refresh`, lưu token mới vào Zustand, rồi tự động thực hiện lại request ban đầu — người dùng không nhận ra gián đoạn.

React Query 5.4 reactquery quản lý toàn bộ server state: fetching, caching tự động, background refetch, và invalidation. Khi YOLO phân tích xong và SSE đẩy sự kiện về trình duyệt, hook `useSSE` gọi `queryClient.invalidateQueries(['media'])` để React Query tự động tải lại dữ liệu media mà không cần reload trang hay polling thủ công.

3.1.3 Recharts và Leaflet.js

Recharts 3.8 recharts xây dựng trên D3.js nhưng cung cấp API khai báo theo phong cách React, phù hợp hơn với kiến trúc component. Hệ thống dùng `AreaChart` cho biểu đồ môi trường (độ sâu, nhiệt độ, áp suất), `LineChart` cho điều hướng (yaw, pitch, roll) và hệ thống (điện áp, pin), `BarChart` cho timeline hoạt động trên Dashboard. Thành phần `<ReferenceLine>` được dùng làm thanh đồng bộ: khi video phát, `currentTime` được chuyển thành timestamp tuyệt đối theo công thức $t_{\text{sync}} = t_{\text{recordedAt}} + t_{\text{video}} \times 1000 \text{ (ms)}$, đặt làm giá trị `x` của `ReferenceLine` để cập nhật đồng bộ với từng sự kiện `timeupdate`. Thành phần `<Brush>` cho phép thu phóng trục thời gian; khi người dùng chọn video trong playlist, vùng Brush tự động nhảy đến khoảng thời gian tương ứng với video đó.

Leaflet.js 1.9 leaflet hiển thị bản đồ Marker tại tọa độ GPS của lượt lặn. Thư viện `react-leaflet` (wrapper React chính thức) không tương thích với React 18.3 do sử dụng lifecycle API cũ, vì vậy Leaflet được khởi tạo trực tiếp qua `useEffect` thuần trên phần tử `<div>` và dọn dẹp khi component unmount. Tile layer dùng `OpenStreetMap` — miễn phí, không cần API key.

3.2 Nền tảng Máy chủ và Lưu trữ (Backend)

3.2.1 Node.js và Express.js

Node.js 20 LTS nodejs với mô hình I/O bất đồng bộ hướng sự kiện (non-blocking Event Loop) xử lý hiệu quả hàng nghìn kết nối HTTP đồng thời với chi phí tài nguyên thấp hơn so với mô hình đa luồng của Spring Boot (Java) hay Django (Python). Đặc điểm này quan trọng khi duy trì kết nối SSE kéo dài: mỗi kết nối SSE là một HTTP response không đóng, và Node.js giữ hàng trăm kết nối song song trên một tiến trình đơn mà không cần tạo thread mới.

Express.js 4.19 expressjs tổ chức backend theo mô hình module: mỗi thực thể nghiệp vụ (Projects, Trips, Media, Sensor, DVL, Sonar, Snapshots, Notifications, Audit) có bộ tệp riêng gồm `routes.js` → `controller.js` → `service.js` → `model.js`. Controller không chứa logic nghiệp vụ — chỉ parse request và gọi Service; Service chứa toàn bộ logic. Response luôn qua hàm `success(res, data)` chuẩn hóa để đảm bảo định dạng JSON nhất quán.

3.2.2 MongoDB Atlas và Mongoose

MongoDB Atlas mongodb (cụm free tier, 512MB) lưu trữ toàn bộ dữ liệu có cấu trúc. Lựa chọn MongoDB thay vì PostgreSQL dựa trên hai lý do chính: (1) Schema cảm biến linh hoạt — một lượt lặn có thể có đủ 7 trường đo (độ sâu, nhiệt độ, áp suất, yaw, pitch, roll, độ ẩm) hoặc chỉ 3 trường bắt buộc, không cần ALTER

TABLE; (2) Mô hình tài liệu phù hợp với cấu trúc phân cấp tự nhiên Project → Trip → SensorData/Media của hệ thống.

Mongoose 8.4 mongoose cung cấp schema validation tại tầng ứng dụng, định nghĩa index kèm khai báo model, và populate để join dữ liệu. Index phức hợp được đặt tại các collection truy vấn thường xuyên:

- `SensorData: { trip: 1, timestamp: -1 }` — truy vấn chuỗi thời gian của một lượt lặn
- `Media: { trip: 1, order: 1 }` — lấy playlist media theo thứ tự kéo-thả
- `Notification: { userId: 1, isRead: 1, createdAt: -1 }` — đếm badge thông báo chưa đọc
- `AuditLog: { userId: 1, createdAt: -1 }` — phân trang nhật ký hoạt động

3.2.3 AWS S3 và cơ chế Presigned URL

Video và ảnh ROV có thể đạt vài trăm MB mỗi tệp. Nếu upload đi qua Node.js, toàn bộ băng thông máy chủ sẽ bị chiếm dụng. Cơ chế **Presigned URL** `awss3` của AWS S3 giải quyết vấn đề này theo luồng bốn bước:

1. Trình duyệt gửi `POST /media/presigned-url` (kèm tên file, MIME type, kích thước) lên Node.js.
2. Node.js dùng AWS SDK ký một `PutObjectCommand` URL với TTL 5 phút và trả về cho trình duyệt.
3. Trình duyệt `PUT` file trực tiếp lên S3 qua URL đã ký — Node.js không nhận một byte dữ liệu nào.
4. Sau khi upload xong, trình duyệt gọi `PATCH /media/:id/confirm` để cập nhật `status = 'ready'`.

Giới hạn kích thước tối đa là 500MB (kiểm tra tại bước 1 trước khi cấp URL). Định dạng được chấp nhận: video (MP4, WebM, MOV, AVI), ảnh (JPEG, PNG, WebP), và PDF. Tệp Sonar nhị phân (.sonar) được upload qua Multer trực tiếp đến Node.js rồi đẩy lên S3, do kích thước nhỏ và cần parse header nhị phân trước khi lưu.

3.2.4 Xác thực JWT, Blacklist Redis và Google OAuth

Hệ thống sử dụng cặp token có vòng đời bất đối xứng theo chuẩn RFC 7519 `jwt spec`:

- **Access Token** (HS256, TTL 15 phút): gửi kèm mọi request dưới dạng `Authorization: Bearer<token>`. Vòng đời ngắn giảm thiểu cửa sổ tấn công nếu token bị lộ.
- **Refresh Token** (TTL 7 ngày): lưu trong MongoDB tại trường `user.refreshToken`. Axios interceptor tự động gọi `POST /auth/refresh` khi nhận lỗi 401, cấp Access Token mới và thực hiện lại request ban đầu mà không làm gián đoạn trải nghiệm người dùng.

Vấn đề của JWT là tính *stateless*: khi người dùng đăng xuất hoặc bị Admin vô hiệu hóa, Access Token vẫn hợp lệ đến khi hết TTL 15 phút. **Redis** giải quyết bằng cơ chế blacklist: khi đăng xuất, token được thêm vào Redis với key pattern `blacklist:{token}` và TTL bằng thời gian sống còn lại. Middleware `authenticate` kiểm tra Redis trước khi chấp nhận mọi request. Nếu Redis không khả dụng, hệ thống vẫn hoạt động bình thường (fail-open) để không ảnh hưởng tính sẵn sàng trong môi trường phát triển.

Ngoài đăng nhập bằng email/mật khẩu, hệ thống hỗ trợ **Google OAuth 2.0 googleoauth** thông qua Passport.js **passportjs**. Khi người dùng nhấn “Đăng nhập bằng Google”, Passport GoogleStrategy tra cứu email trong MongoDB: nếu tài khoản đã tồn tại (đăng ký bằng email) thì gán `googleId` và đăng nhập; nếu chưa tồn tại thì tạo tài khoản mới với `authProvider = 'google'` và role mặc định `viewer`. Tài khoản Google không có trường `password` và không hiển thị tab đổi mật khẩu trong giao diện.

3.3 Xử lý bất đồng bộ và Giao tiếp thời gian thực

Hai tính năng AI của hệ thống có độ trễ cao: phân tích YOLO trên video 30 giây mất từ 30 đến hơn 120 giây tùy phần cứng; tổng hợp AI qua Gemini mất 5–15 giây. Không thể chặn HTTP request trong thời gian đó. Phần này mô tả chi tiết hai thành phần giải quyết bài toán: Bull Queue xử lý tác vụ ngầm và SSE đẩy kết quả về trình duyệt khi hoàn tất.

3.3.1 Bull Queue và cơ chế xử lý bất đồng bộ

Bull 4.16 bullqueue là thư viện hàng đợi tác vụ xây dựng trên Redis, đảm bảo “at-least-once delivery”: nếu worker crash giữa chừng, job được tự động thử lại. Hệ thống khởi tạo bốn hàng đợi độc lập với cấu hình timeout phù hợp đặc thù từng tác vụ (Bảng 3.2).

Bảng 3.2: Cấu hình các hàng đợi Bull Queue trong hệ thống

Tên hàng đợi	Timeout	Thử lại	Tác vụ
ai-summary	60s	3 lần (backoff 2s)	Gọi Gemini API tổng hợp báo cáo
media-analysis	25 phút	1 lần	Phân tích video/ảnh bằng YOLOv8
snapshot-analysis	5 phút	1 lần	Phân tích clip/ảnh bằng chứng
email	60s	3 lần (backoff 2s)	Gửi email thông báo

Luồng hoạt động điển hình khi Operator kích hoạt phân tích YOLO được mô tả như sau: (1) Controller đặt `media.analysisStatus='pending'`, thêm job vào `mediaAnalysisQueue` với payload `{mediaId, mimeType, userId, model, confidence}`, và trả về **HTTP 202** tức thì mà không chờ xử lý. (2) Worker chạy trong tiến trình Node.js song song với Event Loop chính, lấy job từ Redis, gọi YOLO Microservice, và lưu kết quả. (3) Sau khi hoàn tất, worker push SSE đến người dùng. (4) Frontend nhận SSE, gọi `invalidateQueries` để React Query tự động tải lại dữ liệu media.

Hàng đợi `media-analysis` chỉ thử 1 lần vì nếu YOLO timeout ở lần đầu (video quá dài), lần thử lại cũng sẽ timeout lại. Ngược lại, `ai-summary` thử 3 lần với exponential backoff để xử lý lỗi mạng thoáng qua khi gọi Gemini API.

3.3.2 Server-Sent Events (SSE) và cơ chế đẩy thông báo

Sau khi tác vụ ngầm hoàn thành, máy chủ cần thông báo ngay cho trình duyệt mà không cần trình duyệt chủ động hỏi lại (polling). WebSocket `sse_mdn` hỗ trợ giao tiếp hai chiều nhưng yêu cầu nâng cấp giao thức (HTTP Upgrade) và thư viện client bổ sung. Vì bài toán chỉ cần giao tiếp *một chiều* (server → client), **Server-Sent Events (SSE)** qua HTTP/1.1 thông thường được chọn thay thế.

Khi trình duyệt mở kết nối `GET/notifications/stream`, máy chủ trả về response có header `Content-Type:text/event-stream` và giữ kết nối mở vô thời hạn. Mỗi tin nhắn được ghi theo định dạng:

```
data: {"type": "notification", "data": {...}}\n\n
```

Hai ký tự xuống dòng liên tiếp (`\n\n`) đánh dấu kết thúc một sự kiện. EventSource API của trình duyệt tự động tái kết nối khi mất mạng mà không cần lập trình viên xử lý thủ công.

Phía máy chủ, mỗi kết nối SSE được lưu vào cấu trúc `Map<userId, res>` trong bộ nhớ. Nếu người dùng mở tab thứ hai, kết nối cũ bị đóng để tránh nhận thông báo trùng lặp. Khi bất kỳ thành phần nào (Worker AI, Controller Trip, Con-

troller Project) gọi `notificationService.create(userId, ...)`, hàm `pushSSE` tra cứu Map và ghi trực tiếp vào response stream của đúng người dùng đó. Kiến trúc in-memory Map đủ dùng cho hệ thống single-instance trên VPS; khi mở rộng sang multiple instances cần thay bằng Redis Pub/Sub.

3.4 Hệ thống Trí tuệ Nhân tạo

3.4.1 YOLOv8 và Python Microservice

YOLOv8 yolov8 (You Only Look Once phiên bản 8, Ultralytics) là kiến trúc học sâu một giai đoạn (single-stage detector) cho bài toán phát hiện và theo dõi vật thể. Không giống kiến trúc hai giai đoạn như Faster R-CNN (đề xuất vùng nghi vấn trước, phân loại sau), YOLOv8 chia ảnh thành lưới và dự đoán bounding box lẫn nhãn đồng thời trong một lần forward pass duy nhất, giúp tốc độ inference nhanh hơn đáng kể. Biến thể được sử dụng là **YOLOv8n (nano)** với kích thước model xấp xỉ 6MB, có thể chạy trên CPU mà không cần GPU — phù hợp với VPS giá rẻ.

Dịch vụ YOLO được đóng gói thành một **Python FastAPI microservice fastapi** độc lập chạy trên cổng 8000, không chia sẻ bộ nhớ với Node.js. Node.js giao tiếp qua `POST /detect` kèm presigned URL của tệp phương tiện, model và ngưỡng confidence. Kiến trúc microservice này cho phép thay thế hoặc nâng cấp mô hình AI mà không ảnh hưởng đến backend Node.js. Model được cache vào dict Python (`_model_cache`) để tránh đọc lại tệp `.pt` cho mỗi request.

Xử lý ảnh: YOLOv8 chạy `predict()` một lần trên file PNG/JPEG, áp dụng Non-Maximum Suppression (NMS) nội bộ, và trả về danh sách `{name, confidence, bbox}`.

Xử lý video: Phân tích từng frame của video là không thực tế về mặt tài nguyên. Hệ thống áp dụng **chiến lược lấy mẫu thích nghi** dựa trên độ dài đoạn video được xử lý (Bảng 3.3):

Bảng 3.3: Chiến lược lấy mẫu frame thích nghi của YOLO Microservice

Độ dài đoạn	Khoảng cách mẫu	Lý do
< 30 giây	0,2 giây	Video ngắn: mẫu dày để không bỏ sót sự kiện
30 giây – 3 phút	0,5 giây	Cân bằng độ chính xác và thời gian xử lý
> 3 phút	1,0 giây	Video dài: giảm tải CPU, tránh timeout 25 phút

Trên mỗi frame được chọn, hệ thống chạy **ByteTrack bytetrack** (`m.track(tracker="bytetrack.yaml")`) thay vì chỉ `predict`. ByteTrack gán `trackId` ổn định theo thời gian: cùng một vật thể xuất hiện ở nhiều frame

sẽ mang cùng `trackId`, giúp frontend nhảy đến đúng frame khi người dùng click vào nhãn. Sau đó **Cross-class NMS** (ngưỡng $\text{IoU} = 0,3$) loại bỏ bounding box chồng lấp từ các nhãn khác nhau. Kết quả trả về là mảng `[{name, confidence, bbox, frameTime, trackId}]` với `frameTime = frame_idx / fps` (giây thực trong video).

3.4.2 Gemini 2.5 Flash và AI Summary

Gemini 2.5 Flash geminiapi (thư viện `@google/generative-ai 0.24`) được chọn để tổng hợp báo cáo nhiệm vụ bằng ngôn ngữ tự nhiên. So với GPT-4o, Gemini Flash có độ trễ thấp hơn và chi phí token thấp hơn — phù hợp cho văn bản đầu vào khoảng 500–1000 token (thống kê chuyến khảo sát ROV). Toàn bộ quá trình chạy bất đồng bộ qua `aiSummaryQueue`: prompt được xây dựng từ tên project, địa điểm (`locationName`), danh sách trip kèm trạng thái, số file media và số điểm bất thường cảm biến. Gemini sinh ra hai phiên bản báo cáo song song: tiếng Việt (`aiSummary.vi`) và tiếng Anh (`aiSummary.en`), lưu vào MongoDB. Sau khi hoàn tất, worker push SSE đến người kích hoạt.

3.4.3 Phát hiện bất thường Z-Score

Sau khi dữ liệu cảm biến được tải lên, hệ thống tự động phát hiện các điểm bất thường bằng phương pháp Z-Score **zscore_grubbs**. Với mỗi metric m trong tập `{depth, temp, temperature, pressure, voltage, battery_percent, humidity}`, Z-Score của giá trị x_i được tính:

$$z_i = \left| \frac{x_i - \bar{x}}{\sigma} \right|$$

trong đó $\bar{x}$ là giá trị trung bình và σ là độ lệch chuẩn của toàn bộ chuỗi đo lường trong lượt lặn đó. Điểm x_i được đánh dấu bất thường khi $z_i > 2,5$. Ngưỡng 2,5 được chọn thay vì giá trị phổ biến 3,0 để tăng độ nhạy — trong môi trường ROV, ngay cả một spike nhỏ về điện áp hay nhiệt độ cũng có thể báo hiệu sự cố thiết bị. Kết quả trả về cùng mỗi response lấy sensor data: `anomalies: [{index, metric, value, zScore, timestamp}]`; frontend hiển thị bằng dot đỏ trên biểu đồ AreaChart và liệt kê trong panel cảnh báo của `TripDetailPage`.

3.4.4 Reverse Geocoding với OpenStreetMap Nominatim

Khi CSV cảm biến chứa cột `lat/lng`, hệ thống gọi **Nominatim API nominatim** của OpenStreetMap để chuyển tọa độ thành tên địa danh (ví dụ: `16.1031, 108.1403` → “Hòa Cường, Đà Nẵng”). Nominatim là dịch vụ miễn phí, không cần API key, phù hợp với ngân sách đồ án. Hệ thống xử lý tên địa danh tiếng Việt bằng cách bóc tách các tiền tố hành chính (*Phường, Quận, Thành phố,...*) để trả

về tên ngắn gọn. Địa danh này được truyền vào prompt AI Summary thay vì tọa độ số thô, giúp báo cáo AI có ngữ nghĩa hơn. Timeout của API call là 5 giây; nếu Nominatim không phản hồi, hệ thống bỏ qua và giữ `locationName = ""` mà không làm gián đoạn luồng upload.

Chương 3 đã trình bày lý do lựa chọn và cơ chế hoạt động chi tiết của toàn bộ hệ sinh thái công nghệ. Hai cơ chế kỹ thuật then chốt — Bull Queue và SSE — kết hợp tạo ra luồng xử lý phi đồng bộ hoàn chỉnh: tác vụ AI nặng được đưa vào hàng đợi ngay lập tức (HTTP 202), Worker xử lý ngầm không chặn luồng chính, SSE đẩy kết quả về trình duyệt tức thì khi hoàn thành. Kiến trúc này, cùng với microservice YOLO tách rời và tích hợp Gemini AI, là nền tảng cho phần thiết kế chi tiết hệ thống được trình bày trong Chương 4.

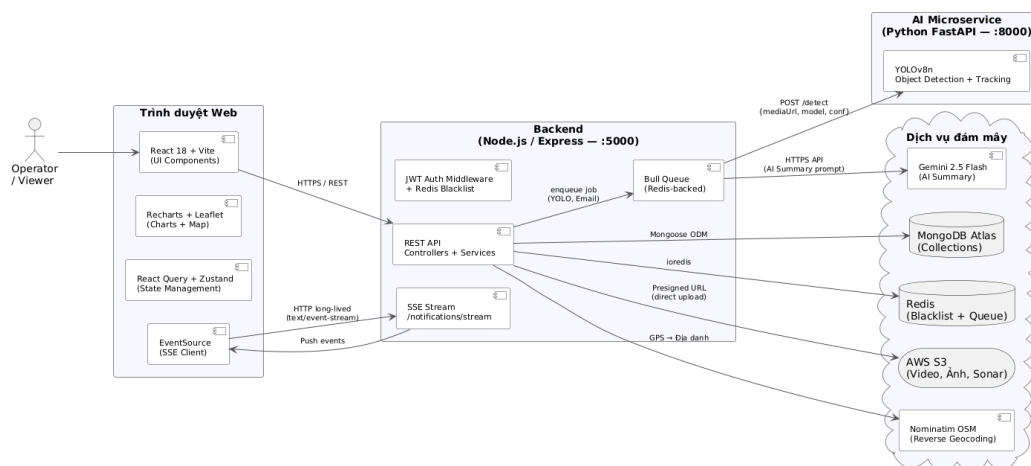
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày quá trình hiện thực hóa hệ thống từ thiết kế kiến trúc đến xây dựng giao diện và kiểm thử. Nội dung gồm bốn phần chính: (1) thiết kế kiến trúc và cơ sở dữ liệu; (2) hiện thực các chức năng cốt lõi kèm minh họa giao diện; (3) kiểm thử chức năng, tải và hiệu năng; (4) mô hình triển khai đề xuất.

4.1 Thiết kế kiến trúc hệ thống

4.1.1 Kiến trúc tổng quan

Hệ thống được thiết kế theo kiến trúc **Client-Server đa tầng**, kết hợp mô hình **Microservice thu nhỏ** để tách biệt bộ phận nhận diện hình ảnh nặng nề khỏi máy chủ chính. Ba tầng chính gồm: (1) tầng trình bày (React SPA), (2) tầng nghiệp vụ (Node.js), và (3) tầng dữ liệu (MongoDB Atlas).



Hình 4.1: Kiến trúc tổng quan hệ thống quản lý ROV

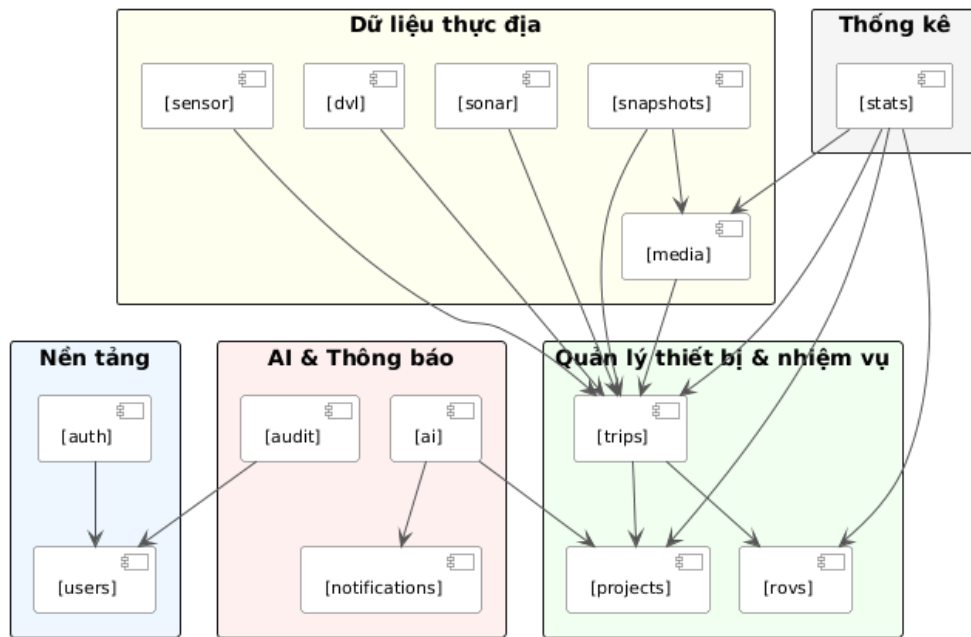
Như Hình 4.1, frontend được phân phối qua Vercel CDN và giao tiếp với backend Node.js thông qua REST API. Backend đảm nhận toàn bộ nghiệp vụ: xác thực JWT, quản lý hàng đợi Bull, xử lý file S3 và điều phối các dịch vụ ngoài (Gemini API, YOLOv8 Microservice, Nominatim). Cơ sở dữ liệu MongoDB Atlas hoạt động độc lập trên cloud.

4.1.2 Sơ đồ phụ thuộc module

Hình 4.2 thể hiện sự phân chia module phía backend theo nguyên tắc *high cohesion, low coupling*. Mỗi thực thể nghiệp vụ (Projects, Trips, Media, Sensor, DVL, Sonar, Snapshots, Notifications, AI, Audit) có bộ tệp độc lập gồm `routes.js`, `controller.js`, `service.js`, `model.js`. Controller không chứa logic

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

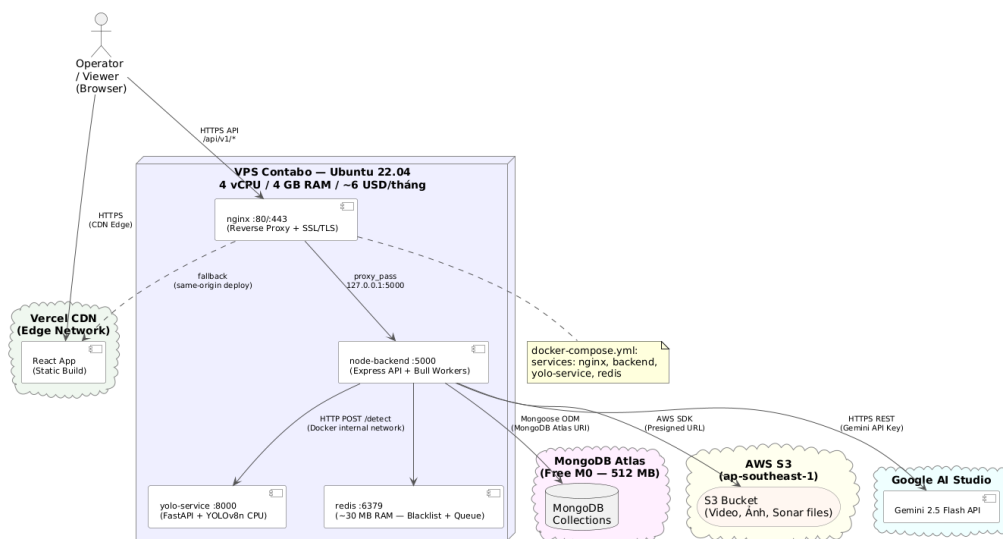
ng nghiệp vụ; Service là trung tâm xử lý duy nhất.



Hình 4.2: Sơ đồ phụ thuộc module phía backend

4.1.3 Mô hình triển khai đề xuất

Do giới hạn phần cứng (YOLOv8 cần khoảng 1,5 GB RAM, không có GPU), hệ thống chưa được triển khai lên VPS trong khuôn khổ đề án. Mô hình triển khai đề xuất (Hình 4.3) sử dụng Docker Compose để đóng gói bốn dịch vụ trên một VPS duy nhất: nginx (reverse proxy), node-backend (Express API + Bull Workers), yolo-service (FastAPI + YOLOv8n), redis (JWT blacklist + hàng đợi). Frontend được deploy riêng trên Vercel CDN; cơ sở dữ liệu ở MongoDB Atlas; file media trên AWS S3.



Hình 4.3: Mô hình triển khai đề xuất trên VPS

Bảng 4.1: Mô tả các collection chính trong MongoDB

Collection	Trường quan trọng	Mô tả
users	email, role, googleId, isActive	Người dùng; role: viewer/operator/admin
rovs	name, serialNumber, status	Thiết bị ROV; status: active/maintenance/retired
projects	rov, status, aiSummary	Chuyến khảo sát; status: planned/ongoing/completed/cancelled
trips	project, status, gpsLocation, sensorCount	Lần lặn; status: pending/running/done/failed
sensordatas	trip, timestamp, depth, temp, sourceFile	Chuỗi thời gian cảm biến; index {trip, timestamp}
dvldatas	trip, ts, x, y, z, sourceFile	Quỹ đạo DVL theo thời gian
sonarfiles	trip, filename, frameCount, durationMs	Metadata file sonar nhị phân trên S3
media	trip, type, labels, analysisStatus	Video/ảnh; labels từ YOLO
snapshots	trip, type, imageS3Key, aiLabels	Ảnh frame hoặc clip bằng chứng
notifications	userId, type, isRead, link	Thông báo in-app qua SSE
auditlogs	action, entity, userId, details	Nhật ký hoạt động admin

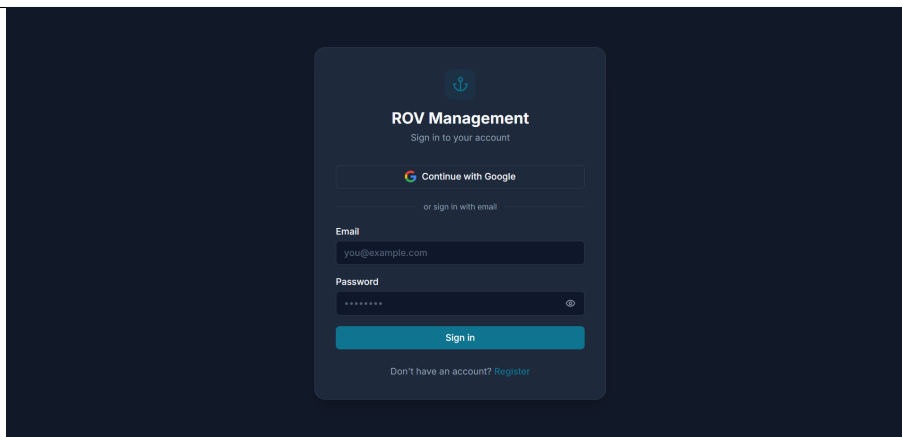
Index phức hợp được đặt tại các collection truy vấn thường xuyên:

`SensorData{trip,timestamp,-1},Media{trip,order},Notification{user`

4.3 Hiện thực các chức năng chính

4.3.1 Xác thực và phân quyền

Màn hình đăng nhập (Hình 4.5) hỗ trợ hai luồng: đăng nhập bằng email/mật khẩu và đăng nhập bằng tài khoản Google qua OAuth 2.0. Sau khi xác thực thành công, hệ thống cấp Access Token (TTL 15 phút) và Refresh Token (TTL 7 ngày). Axios interceptor tự động làm mới token khi nhận lỗi 401, đảm bảo trải nghiệm không gián đoạn.



Hình 4.5: Màn hình đăng nhập — hỗ trợ Google OAuth

Hệ thống phân quyền RBAC (Bảng 4.2) được thực thi tại middleware `authorize(roles)` trên mọi route. Frontend ẩn các nút thao tác không phù hợp với role hiện tại, nhưng backend vẫn kiểm tra độc lập để tránh bypass qua API trực tiếp.

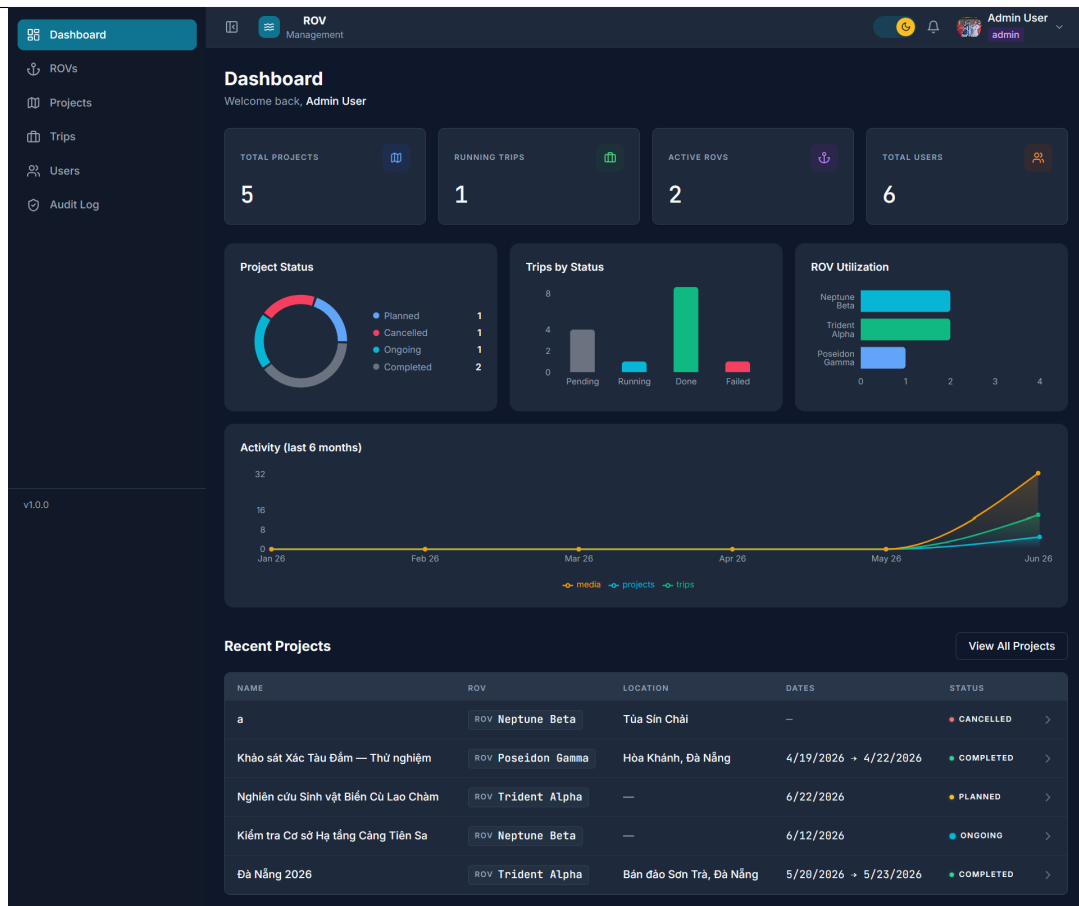
Bảng 4.2: Ma trận phân quyền RBAC của hệ thống

Chức năng	admin	operator	viewer
Quản lý người dùng	✓	×	×
Xem Audit Log	✓	×	×
Xóa Project / Trip	✓	×	×
Xóa ROV / Media	✓	×	×
Tạo/sửa Project, Trip	✓	✓	×
Tạo/sửa ROV	✓	✓	×
Upload sensor / media	✓	✓	×
Xóa dữ liệu cảm biến / DVL / Sonar	✓	✓	×
Tạo/xóa Bằng chứng (Evidence)	✓	✓	×
Kích hoạt AI Summary / YOLO	✓	✓	×
Xem toàn bộ dữ liệu	✓	✓	✓
Export CSV/PDF	✓	✓	×

4.3.2 Dashboard và thống kê tổng quan

Dashboard (Hình 4.6) hiển thị bức tranh tổng quan hệ thống qua bốn thẻ KPI và ba biểu đồ Recharts, được tải song song qua bảy MongoDB aggregation. Thẻ KPI bao gồm: tổng số Project, số Trip đang chạy, số ROV hoạt động và tổng người dùng. Biểu đồ gồm: Project Status (donut), Trips by Status (bar), ROV Utilization (horizontal bar) và Activity Timeline 6 tháng.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



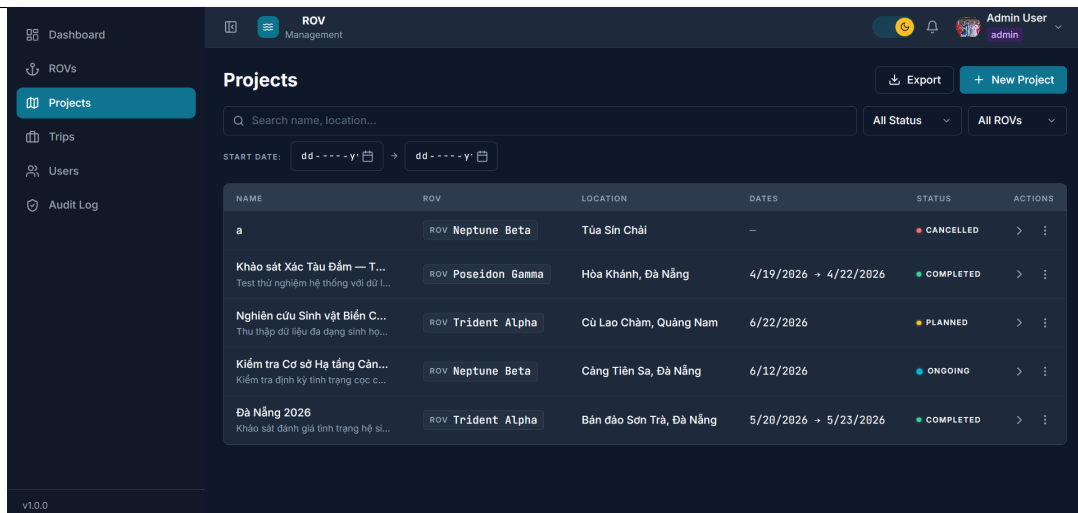
Hình 4.6: Dashboard — KPI cards và biểu đồ thống kê tổng quan

4.3.3 Quản lý Project và Trip

a, Danh sách Projects

Trang Projects (Hình 4.7) hiển thị danh sách dạng bảng với đầy đủ tính năng: tìm kiếm theo tên, lọc theo trạng thái và khoảng thời gian, phân trang, xuất CSV/PDF. Mỗi hàng có nút mở rộng để xem nhanh danh sách Trip và nút điều hướng đến trang chi tiết.

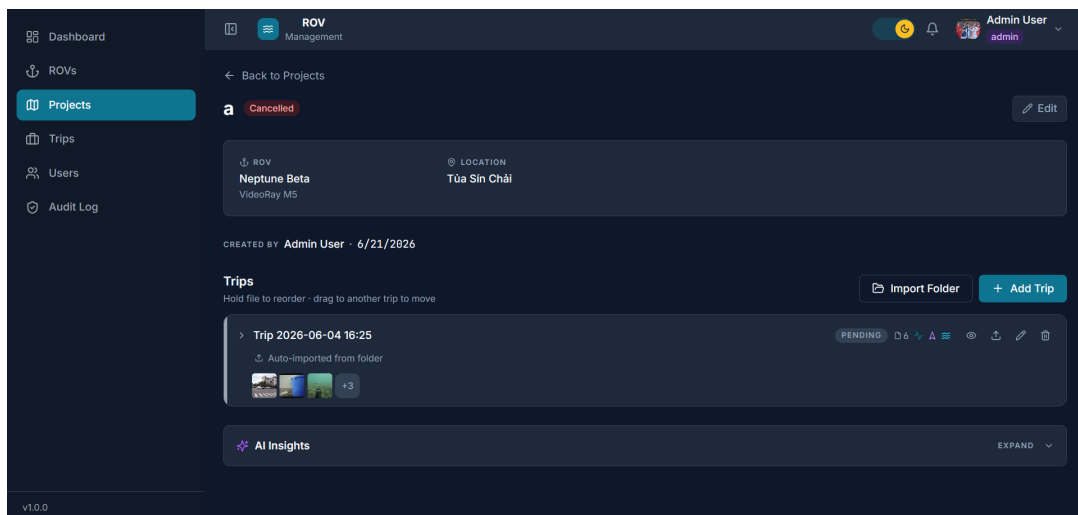
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.7: Trang danh sách Projects — filter, search và export

b, Chi tiết Project

ProjectDetailPage (Hình 4.8) hiển thị toàn bộ thông tin project và danh sách Trip con. Mỗi TripCard khi mở rộng hiện phần DATA FILES với tên file cụ thể (sensor CSV, DVL JSON, sonar), cho phép xóa từng file riêng lẻ, phía dưới là MediaGallery. Khu vực AI Summary hiện nội dung tóm tắt song ngữ (vi/en) sau khi Operator kích hoạt Generate.

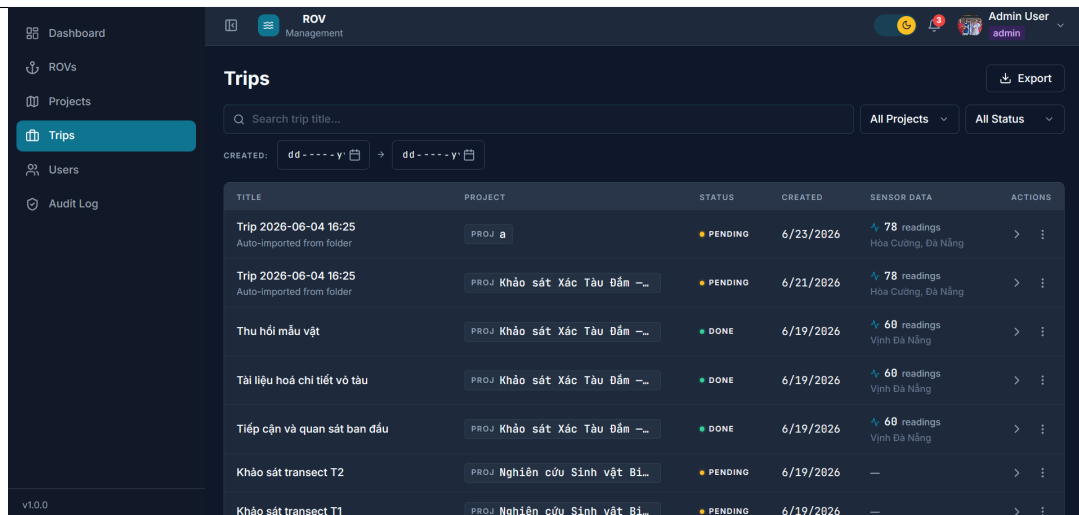


Hình 4.8: ProjectDetailPage — TripCard expanded với DATA FILES và AI Summary

c, Danh sách Trips

Trang Trips (Hình 4.9) cho phép lọc toàn bộ lượt lặn theo trạng thái và ngày, độc lập với Project. Mỗi hàng có badge trạng thái màu (pending: xám, running: xanh dương, done: xanh lá, failed: đỏ) và số lượng sensor/media.

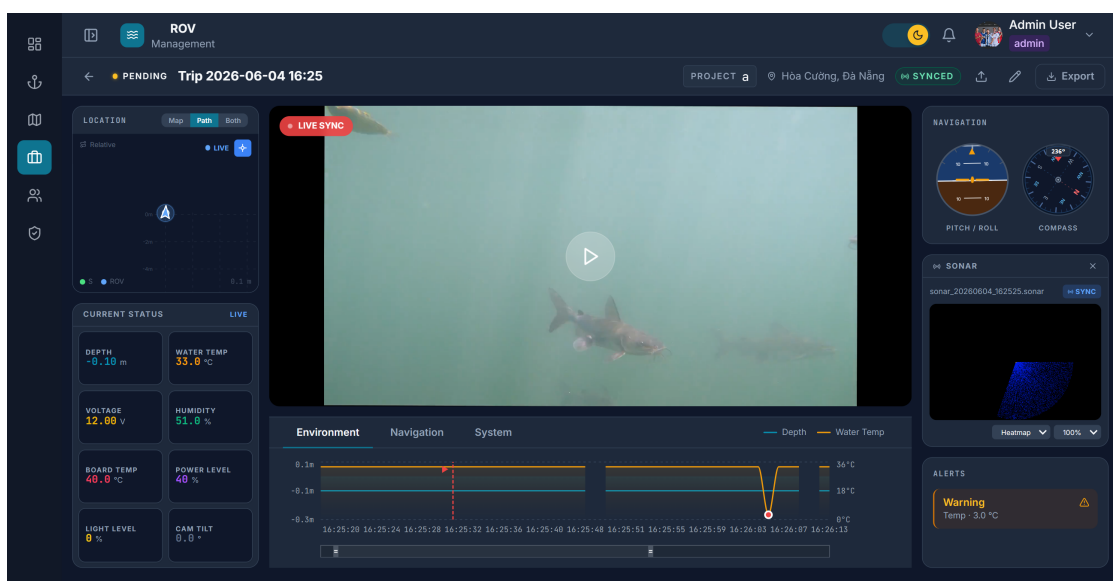
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.9: Trang danh sách Trips — filter theo status và date range

4.3.4 Giao diện Cockpit TripDetailPage

TripDetailPage là tính năng cốt lõi của hệ thống, được thiết kế theo phong cách “cockpit” (Hình 4.10): layout 3 cột không cuộn trang, tối ưu cho màn hình 1080p.



Hình 4.10: TripDetailPage Cockpit — layout 3 cột đầy đủ tính năng

Cột trái (w-56) hiển thị: bản đồ Leaflet với Marker GPS lần lặn, ba thẻ KPI (độ sâu trung bình, nhiệt độ, áp suất) và mục DATA FILES liệt kê tên từng file dữ liệu kèm nút xóa riêng lẻ.

Cột giữa (flex-1) là trình phát video nền đen, phía trên có thanh gradient chứa tên file và các nút điều khiển AI: “Detect/Hide” bật/tắt bounding box YOLO, “Analyze” mở popover cài đặt model/confidence, “” chụp frame ảnh, “” ghi clip, “Evidence N” mở Evidence Panel. Khi video có recordedAt, badge “LIVE SYNC”

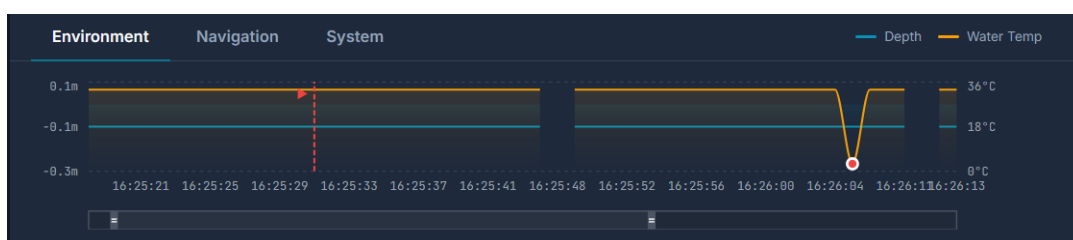
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

xuất hiện tại góc và thanh đồ ReferenceLine trên biểu đồ bên dưới di chuyển theo từng giây phút.

Cột phải (w-56) có đồng hồ ArtificialHorizon và CompassRose dạng SVG kiểu hàng không, animate theo dữ liệu yaw/pitch/roll từ CSV. Phần Alerts hiển thị card cảnh báo màu hổ phách cho từng điểm bất thường.

Dưới cùng là ba tab biểu đồ: Environment (depth/temp/pressure), Navigation (yaw/pitch/roll), System (voltage/battery/humidity), kèm Brush zoom, legend toggle và nút mở rộng.

Biểu đồ Environment (Hình 4.11) làm nổi bật các điểm bất thường bằng dot đỏ, kết quả từ thuật toán Z-Score tự cài đặt với ngưỡng $|z| > 2,5$.

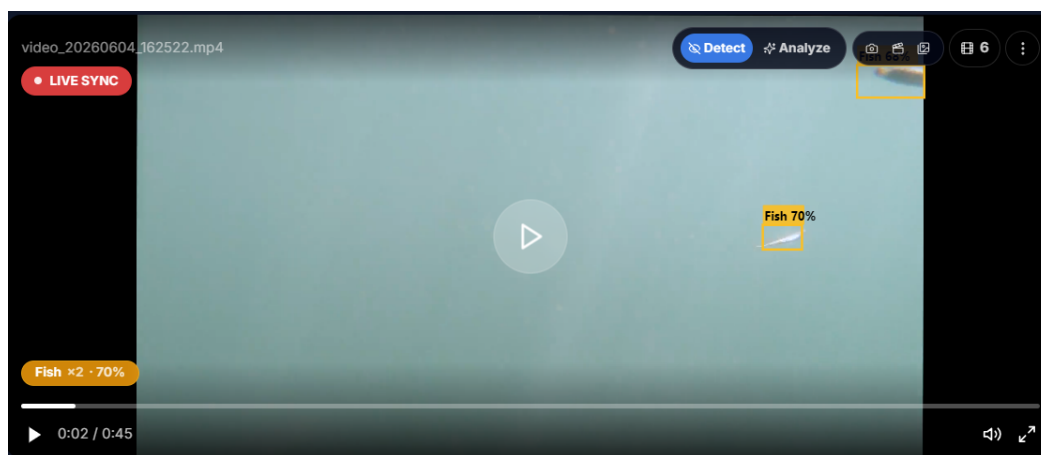


Hình 4.11: Bottom chart tab Environment — dot đỏ đánh dấu điểm bất thường Z-Score

4.3.5 Nhận diện vật thể YOLOv8 và Evidence System

a, Bounding box YOLO

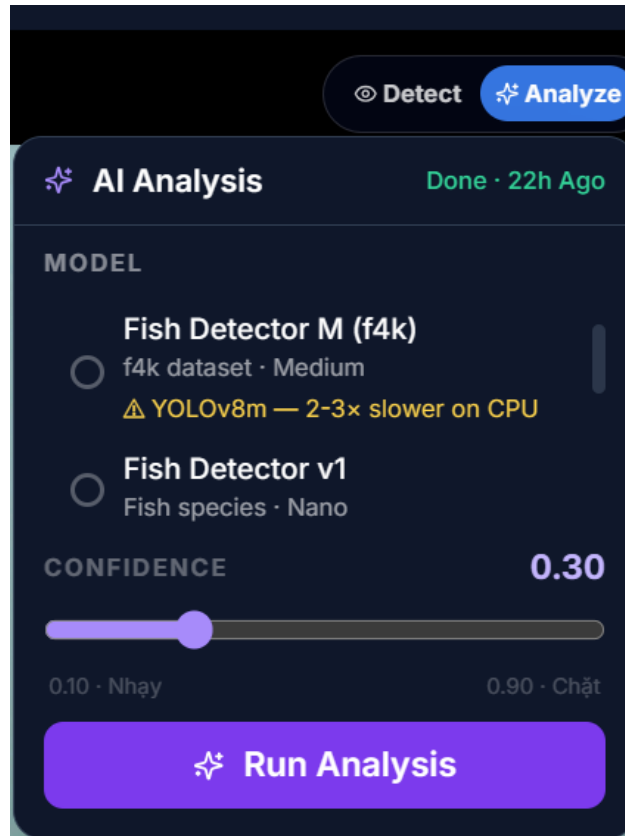
Khi Operator nhấn “Detect”, hệ thống hiển thị bounding box SVG chồng lên video/ảnh (Hình 4.12). Mỗi box có nhãn tên vật thể và độ tin cậy; badge xanh nếu confidence > 80%, vàng nếu 50–80%. Đối với video đã phân tích per-frame, box di chuyển theo thời gian phát và badge bar cập nhật theo frame hiện tại.



Hình 4.12: Bounding box YOLO overlay trên ảnh/video ROV

b, AI Analyze Popover

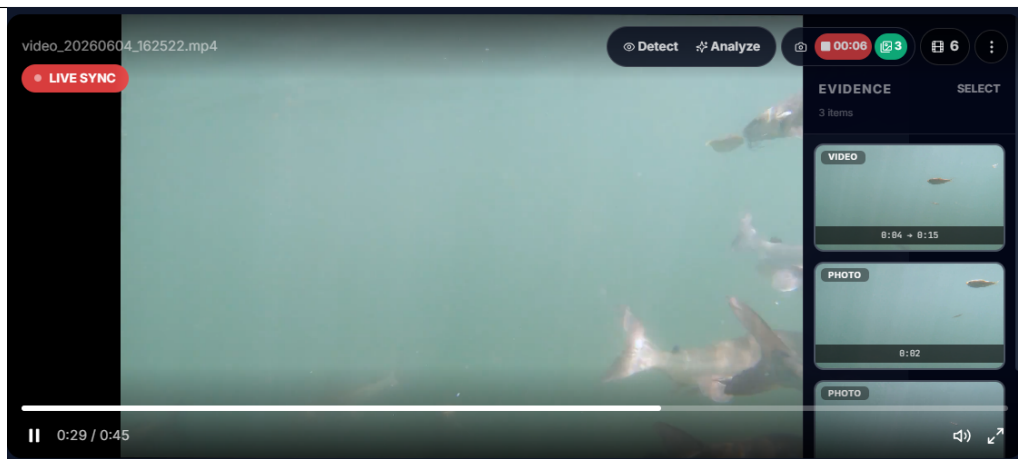
Operator chọn model và điều chỉnh ngưỡng confidence qua popover nhỏ (Hình 4.13) trước khi kích hoạt phân tích. Hệ thống hỗ trợ nhiều model; model nặng (YOLOv8m) hiển thị warning màu vàng. Yêu cầu phân tích được đưa vào Bull Queue và trả về ngay HTTP 202, tránh block giao diện.



Hình 4.13: AI Analyze Popover — chọn model và ngưỡng confidence trước khi phân tích YOLO

c, Evidence System

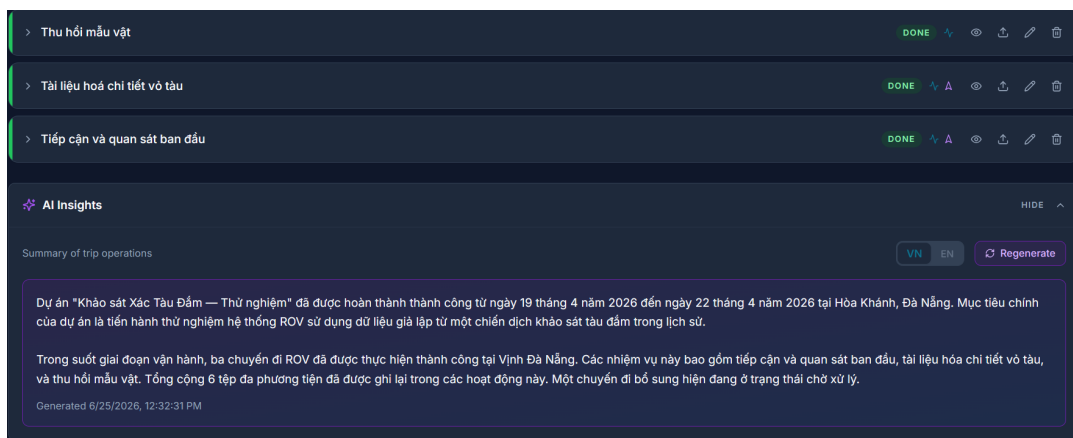
Operator có thể chụp frame ảnh () hoặc ghi đoạn clip () làm bằng chứng trong khi xem video. Evidence Panel (Hình 4.14) hiển thị thumbnails kiểu single-column; click vào thumb mở EvidenceViewer overlay với đầy đủ điều khiển phát lại và bounding box.



Hình 4.14: Evidence Panel — PHOTO/CLIP thumbnails và EvidenceViewer overlay

4.3.6 Tổng hợp báo cáo bằng AI (Gemini)

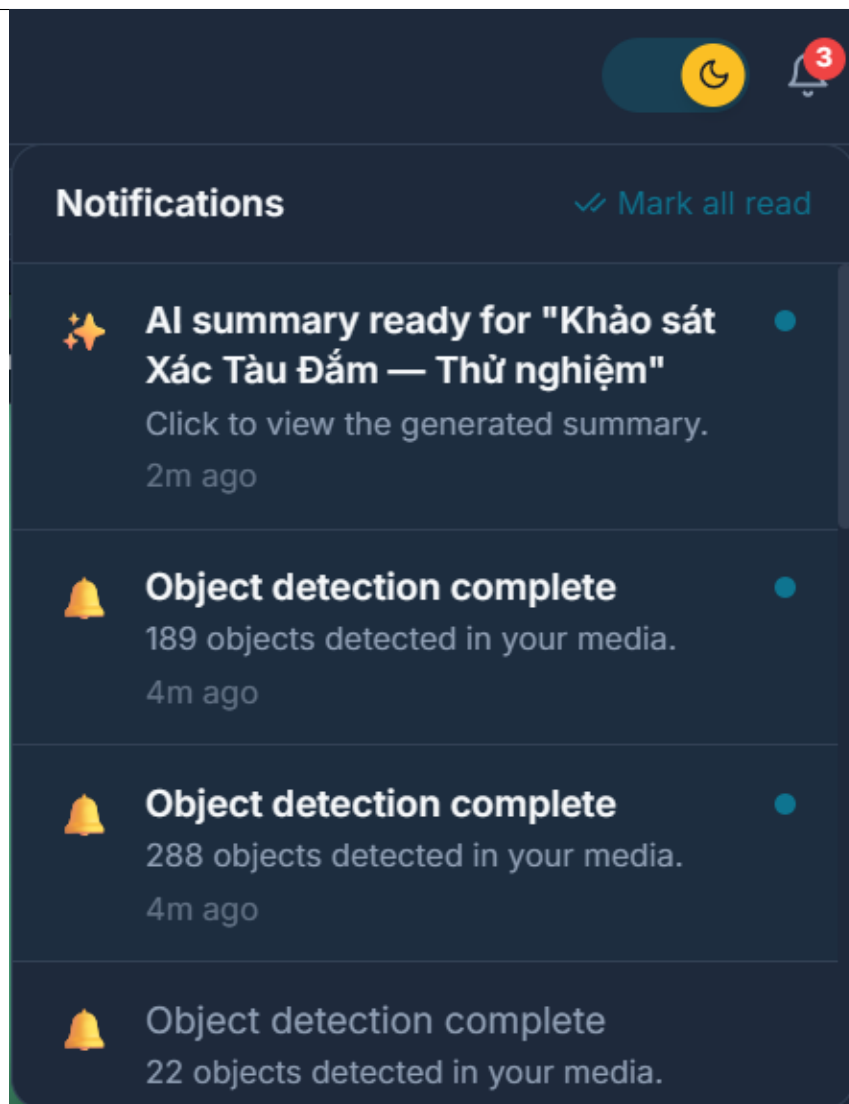
Khi Project đạt trạng thái `completed`, Operator/Admin có thể kích hoạt “Generate AI Summary”. Yêu cầu được đưa vào hàng đợi `ai-summary` và trả về HTTP 202 tức thì. Sau 5–15 giây, Gemini 2.5 Flash sinh ra báo cáo song ngữ (tiếng Việt và tiếng Anh) từ prompt tổng hợp thông tin project, trip và điểm bất thường. Kết quả được SSE đẩy về frontend và hiển thị trong section AI Summary (Hình 4.15).



Hình 4.15: AI Summary — báo cáo song ngữ được tổng hợp bởi Gemini 2.5 Flash

4.3.7 Thông báo realtime và quản lý người dùng

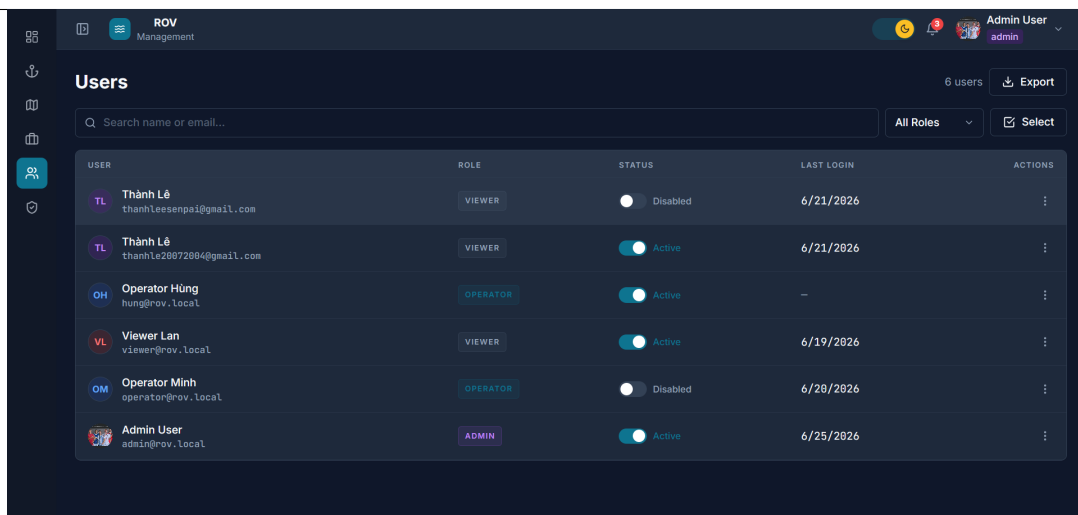
Hệ thống đẩy thông báo realtime qua SSE mỗi khi có sự kiện quan trọng (trip chuyển sang `done/failed`, AI Summary hoàn tất). Bell icon trên navbar hiển thị badge số chưa đọc; click mở dropdown 15 thông báo gần nhất (Hình 4.16).



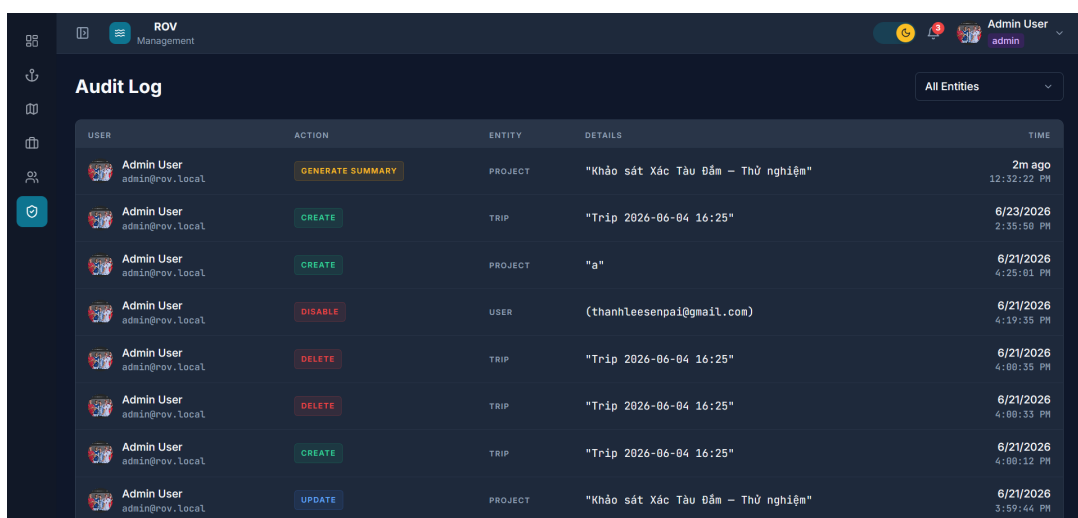
Hình 4.16: Notification bell dropdown — thông báo realtime qua SSE

Trang quản lý người dùng (Hình 4.17, chỉ hiện với Admin) cung cấp: tìm kiếm, lọc theo role, đổi role đơn lẻ hoặc bulk, kích hoạt/vô hiệu hóa tài khoản, xuất CSV. Mọi thao tác admin được ghi vào Audit Log (Hình 4.18).

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.17: Trang quản lý người dùng (Admin) — bulk operations và role management



Hình 4.18: Audit Log — nhật ký hoạt động admin theo timeline

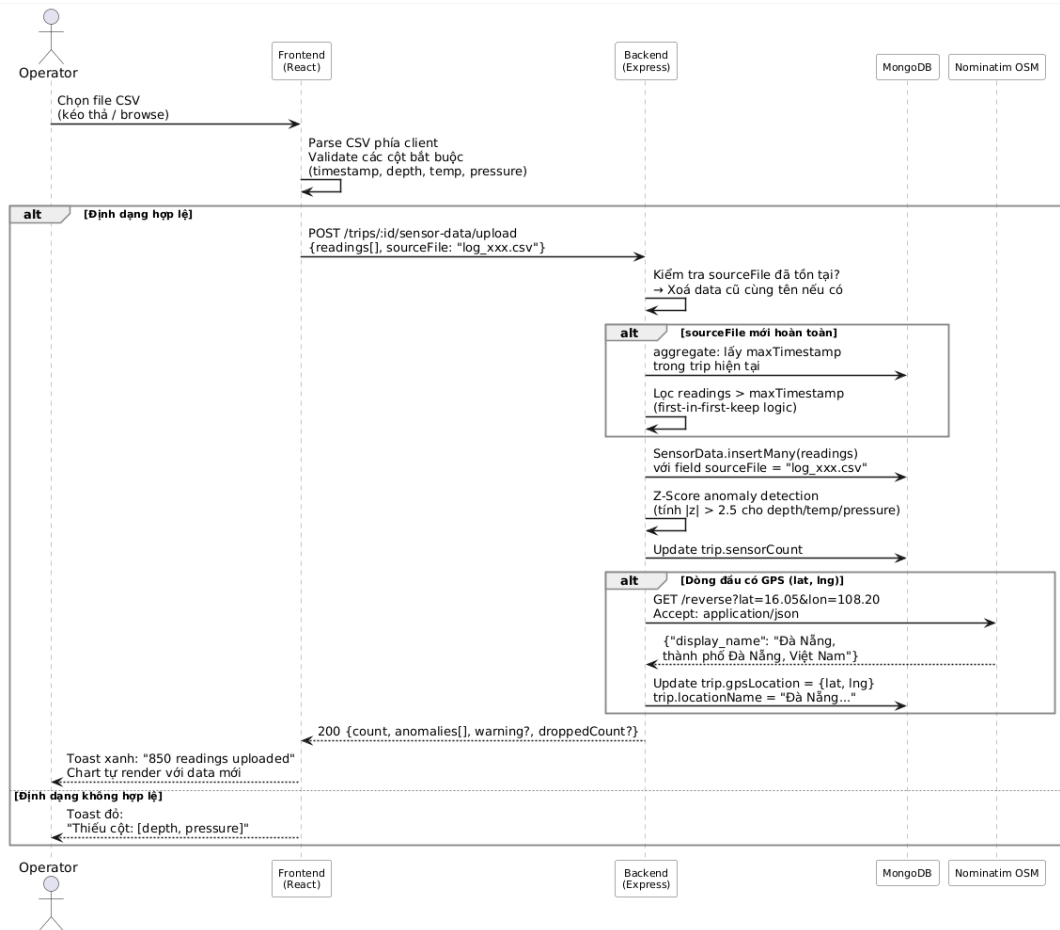
4.3.8 Sơ đồ tuần tự các luồng nghiệp vụ chính

Hai sơ đồ tuần tự dưới đây mô tả chi tiết tương tác giữa các thành phần hệ thống cho hai luồng nghiệp vụ quan trọng nhất: tải lên dữ liệu cảm biến và phân tích YOLO bất đồng bộ.

a, Luồng tải lên dữ liệu cảm biến CSV

Hình 4.19 mô tả luồng tải lên dữ liệu cảm biến từ khi Operator chọn file đến khi dữ liệu được lưu vào MongoDB và bản đồ GPS được cập nhật.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



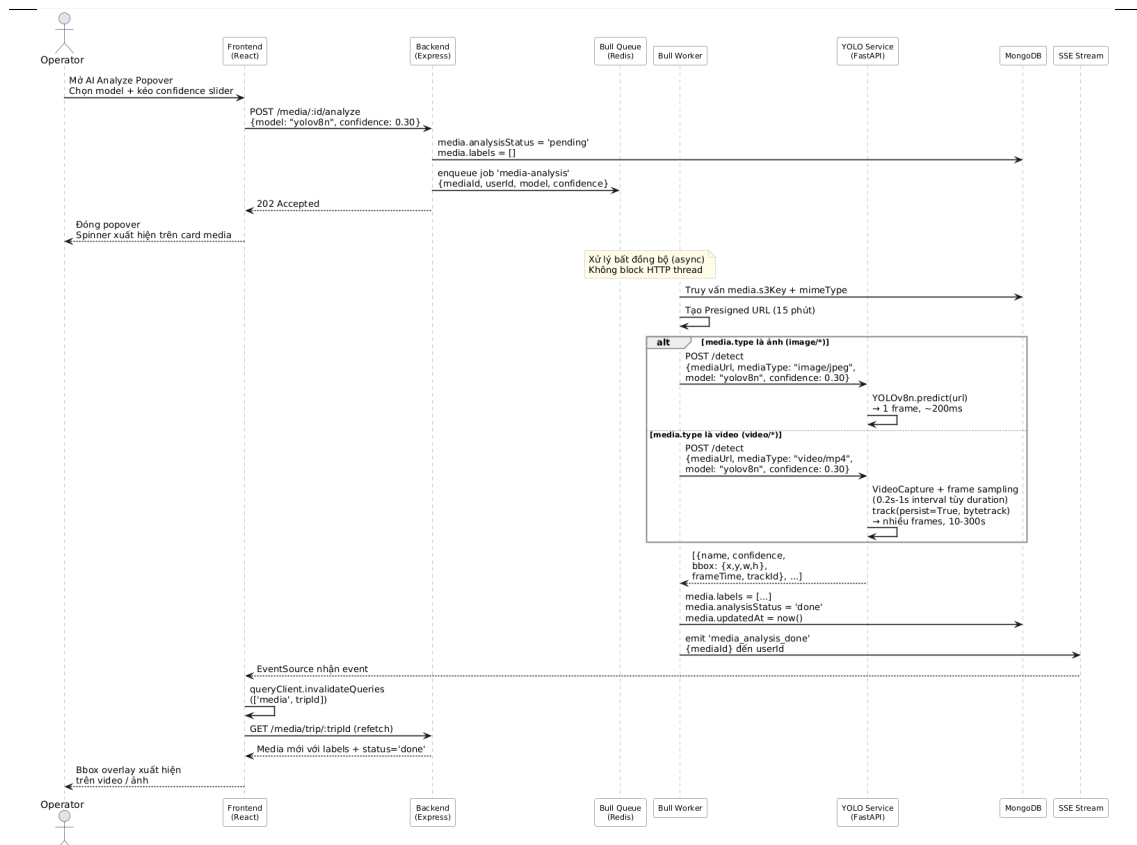
Hình 4.19: Sơ đồ tuần tự — luồng tải lên file CSV cảm biến

Luồng gồm sáu bước chính: (1) Frontend parse file CSV phía client, validate định dạng (bắt buộc timestamp và depth, các trường còn lại tùy chọn) trước khi gửi lên server; (2) `POST/trips/:id/sensor-data` gửi mảng readings kèm tên file gốc (`sourceFile`); (3) Controller kiểm tra `sourceFile` đã tồn tại trong DB chưa — nếu có thì xóa data của file đó trước (re-upload), nếu là file mới thì kiểm tra overlap theo timestamp lớn nhất hiện có và cắt bỏ dữ liệu trùng; (4) `SensorData.insertMany()` ghi batch toàn bộ readings hợp lệ; (5) Nếu dòng đầu tiên có tọa độ `lat/lon`, hệ thống gọi Nominatim reverse geocoding để lấy tên địa danh và cập nhật `trip.gpsLocation+trip.locationName`; (6) Trả về `{count, total, warning?}` — frontend hiển thị toast xanh (thành công), vàng (`overlap_trimmed` hoặc `file_skipped`).

b, Luồng phân tích YOLO bất đồng bộ qua Bull Queue

Hình 4.20 mô tả luồng phân tích nhận diện vật thể: Operator kích hoạt từ `AIAnalyzePopover`, backend trả ngay HTTP 202, Bull Worker xử lý nền và SSE đẩy kết quả về khi xong.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.20: Sơ đồ tuần tự — luồng phân tích YOLO bất đồng bộ qua Bull Queue

Luồng gồm bảy bước: (1) Operator chọn model và ngưỡng confidence trong popover, nhấn “Run Analysis”; (2) `POST /media/:id/analyze {model, confidence}` đến backend; (3) `mediaService.enqueueAnalysis()` cập nhật `media.analysisStatus='pending'`, `labels = []` rồi thêm job vào hàng đợi `mediaAnalysisQueue`; (4) Backend trả HTTP 202 ngay lập tức — giao diện người dùng không bị chặn; (5) Bull Worker lấy job: tạo presigned GET URL (TTL 10 phút) để YOLO service tải file từ S3, gọi `POST yolo-service:8000/detect {mediaUrl, model, confidence}`; (6) YOLO service tải file từ S3, chạy inference (ảnh: 1 lần, video: lấy mẫu thích nghi 0,2–1,0 giây/khung tùy độ dài), trả mảng `labels[] {name, confidence, bbox, frameTime, trackId}`; (7) Worker lưu kết quả, đặt `analysisStatus='done'`, gọi `notifService.create()` để đẩy SSE về client — frontend nhận event, invalidate query `['media']` và labels xuất hiện mà không cần tải lại trang.

Thiết kế bất đồng bộ này đảm bảo trải nghiệm người dùng mượt mà: phân tích video dài có thể mất 5–15 phút nhưng giao diện không bị đóng băng. Nếu YOLO service không phản hồi (ECONNREFUSED), Worker ghi `analysisStatus='failed'` mà không crash queue.

4.4 Kiểm thử hệ thống

4.4.1 Kiểm thử chức năng (Functional Test)

Script `functional-test.js` được xây dựng để tự động hóa 37 ca kiểm thử bao phủ đầy đủ các nhóm chức năng:

- **Xác thực (7 ca):** đăng nhập đúng, sai mật khẩu, email không tồn tại, thiếu field, GET `/auth/me` với token hợp lệ/không có/sai.
- **Phân quyền RBAC (6 ca):** Viewer không tạo/xóa được Project, Trip; thao tác không xác thực trả 401.
- **Validation (5 ca):** thiếu field bắt buộc, sai ObjectId format, projectId không tồn tại → 404, sensor readings không phải array, array rỗng.
- **CRUD (11 ca):** tạo/đọc/sửa/xóa Project và Trip, cascade delete, xác nhận 404 sau khi xóa.
- **Phân trang (6 ca):** limit đúng, cấu trúc response paginated, filter theo status, page không tồn tại → data rỗng.
- **Kiểm thử hồi quy (2 ca):** POST `/projects/:id/trips` với projectId không tồn tại phải trả 404.

```

PS D:\Code\rov-management\backend> node src/scripts/functional-test.js
Functional Test – ROV Management API
=====
-
AUTH
  ✓ Login với đúng credentials → 200
  ✓ Response có accessToken
  ✓ Login sai password → 401
  ✓ Login email không tồn tại → 401

  ✓ Login thiếu password → 400/401
  ✓ GET /auth/me với token hợp lệ → 200
  ✓ /auth/me trả về đúng email
  ✓ GET /auth/me không có token → 401
  ✓ GET /auth/me token không hợp lệ → 401

RBAC – Phân quyền
  ✓ Viewer login OK
  ✓ Viewer POST /projects → 403
  ✓ Viewer DELETE /trips/:id → 403
  ✓ Viewer GET /projects → 200
  ✓ Viewer batch upload → 403
  ✓ Không có token GET /projects → 401

VALIDATION – Kiểm tra đầu vào
  ✓ POST /projects thiếu name → 400/422
  ✓ POST /projects với rovid sai format → 400/422/500
  ✓ POST /projects/:id/trips với id không tồn tại → 404
  ✓ Upload sensor-data readings không phải array → 400
  ✓ Upload sensor-data array rỗng → 400

CRUD – Projects & Trips
  ✓ GET /rovs → 200
  ✓ POST /projects → 201
  ✓ GET /projects/:id → 200
  ✓ GET /projects/:id trả đúng name
  ✓ PATCH /projects/:id → 200
  ✓ PATCH cập nhật status đúng
  ✓ POST /projects/:id/trips → 201
  ✓ GET /trips/:id → 200
  ✓ DELETE /trips/:id → 200
  ✓ DELETE /projects/:id → 200
  ✓ GET project đã xóa → 404

PAGINATION & FILTER
  ✓ GET /projects?limit=2 → 200
  ✓ Trả về đúng cấu trúc page
  ✓ Số item ≤ limit=2
  ✓ GET /projects?status=completed → 200
  ✓ Tất cả kết quả có status=completed
  ✓ GET /projects?page=999 → 200 với data rỗng

=====
Kết quả: 37/37 passed (100%)
✓ TẤT CẢ PASS – API hoạt động đúng
PS D:\Code\rov-management\backend>

```

Hình 4.21: Kết quả kiểm thử chức năng — 37/37 ca đạt (100%)

Kết quả Hình 4.21 cho thấy 37/37 ca kiểm thử đều PASSED (tỉ lệ 100%). Trong quá trình viết test, phát hiện và sửa một lỗi thực tế: endpoint `POST /projects/:id/trips` không kiểm tra project tồn tại, trả về 201 thay vì 404 khi projectId sai.

4.4.2 Kiểm thử tải (Load Test)

Script `load-test.js` bắn 20 request đồng thời $\times$ 3 vòng = 60 request/endpoint lên backend đang chạy local với MongoDB Atlas (Bảng 4.3).

Bảng 4.3: Kết quả kiểm thử tải — 20 concurrent $\times$ 3 rounds (60 requests/endpoint)

Endpoint	avg	p50	p95	Errors
POST /auth/login	1312ms	8ms	4101ms	41/60
GET /auth/me	132ms	82ms	492ms	0/60
GET /projects	474ms	456ms	934ms	0/60
GET /trips	458ms	252ms	988ms	0/60
GET /rovs	205ms	168ms	540ms	0/60
GET /stats/overview	825ms	887ms	1830ms	0/60
GET /trips/:id/sensor-data	327ms	261ms	523ms	0/60
GET /trips/:id/dvl	737ms	810ms	885ms	0/60
GET /media/trip/:id	377ms	241ms	759ms	0/60
GET /notifications	525ms	645ms	855ms	0/60

Phân tích kết quả:

- **POST /auth/login** có 41/60 lỗi là hoạt động đúng thiết kế — rate limiter Redis chặn 20 request/15 phút để phòng chống brute-force. Các request đến sau ngưỡng trả về 429 Too Many Requests.
- **GET /auth/me** với p95 = 492ms là kết quả tốt, phản ánh MongoDB Atlas (free tier) có độ trễ mạng Singapore-Vietnam khoảng 50–70ms.
- **GET /stats/overview** chậm nhất (avg 825ms, p95 1830ms) do thực hiện 7 aggregation song song. Trong môi trường production với index đầy đủ và VPS gần MongoDB, thời gian này giảm đáng kể.
- Tất cả endpoint đọc dữ liệu hoạt động ổn định không có lỗi ở tải 20 concurrent.

4.4.3 Kiểm thử hiệu năng Lighthouse

Công cụ Lighthouse (Chrome DevTools, chế độ Desktop, throttling mô phỏng) được chạy trên ba trang đại diện của ứng dụng (Bảng 4.4).

Bảng 4.4: Kết quả kiểm thử Lighthouse — Desktop mode

Trang	Perf.	A11y	BP	SEO	FCP	LCP
Dashboard	98	94	96	100	0,8s	0,9s
ProjectsPage	95	81	100	100	0,7s	1,4s
TripDetailPage	88	89	100	100	0,7s	1,9s

Nhận xét:

- **Dashboard** đạt Performance 98 — xuất sắc. FCP 0,8s và LCP 0,9s cho thấy trang tải rất nhanh nhờ kiến trúc React SPA với Vite build tối ưu. TBT chỉ 90ms chứng tỏ main thread ít bị chặn.
- **ProjectsPage** đạt Performance 95. Accessibility 81 do một số button thiếu accessible name và input date không có label — vấn đề cần cải thiện trong phiên bản tiếp theo.
- **TripDetailPage** đạt Performance 88 do LCP 1,9s: ảnh/video phải tải từ AWS S3 (Singapore) mất thêm 40–50ms RTT so với localhost. Đây là giới hạn về hạ tầng, không phải tối ưu code. TBT 100ms vẫn trong ngưỡng tốt.
- **Best Practices 100** trên hai trang kỹ thuật phản ánh code không có lỗi console, không dùng API deprecated, không có third-party cookies.
- **SEO 100** trên tất cả trang nhờ `<title>`, `<meta description>`, hreflang hợp lệ và tất cả link có thể crawl được.

4.5 Mô hình triển khai (đề xuất)

Do yêu cầu phần cứng của YOLOv8 (1,5 GB RAM), hệ thống chưa triển khai lên VPS trong phạm vi đề án này. Mô hình triển khai đề xuất được thiết kế để dễ dàng thực hiện khi có hạ tầng phù hợp.

Cấu hình đề xuất:

- **Frontend:** Vercel (CDN toàn cầu, deploy tự động từ git push).
- **Backend VPS:** Contabo 4 vCPU / 4 GB RAM / 6 USD/tháng, chạy Ubuntu 22.04.
- **Docker Compose:** 4 service — `nginx:alpine` (reverse proxy + SSL), `node-backend` (Express + Bull Workers), `yolo-service` (FastAPI + YOLOv8n CPU), `redis:alpine` (30MB RAM).
- **Database:** MongoDB Atlas M0 (free tier, 512MB).
- **Storage:** AWS S3 ap-southeast-1 (pay-per-use, gần như miễn phí với traffic demo).
- **AI:** Gemini 2.5 Flash qua Google AI Studio API (free tier 1500 request/ngày).

Cách triển khai: `git clone` lên VPS, tạo `.env`, chạy `docker compose up -d`. Nginx tự động cấp SSL/TLS qua Let's Encrypt và điều hướng: `/api/v1/*` → Node.js:5000, còn lại → Vercel fallback.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Ước tính chi phí hàng tháng: VPS \$6 + MongoDB Atlas \$0 + AWS S3 <\$1 + Gemini \$0 = **\$7/tháng** — hoàn toàn khả thi cho môi trường demo và thử nghiệm thực địa.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này tập trung trình bày chi tiết các giải pháp công nghệ mang tính cốt lõi và là đóng góp quan trọng nhất của đề án. Thay vì tập trung vào các chức năng quản lý thông tin thông thường (CRUD), nội dung sẽ đi sâu phân tích cách thức giải quyết ba bài toán khó khăn nhất gặp phải trong quá trình phát triển hệ thống quản lý dữ liệu ROV: (1) Thuật toán đồng bộ hóa thời gian giữa đa phương tiện và dữ liệu cảm biến, (2) Kiến trúc xử lý bất đồng bộ tích hợp mô hình nhận diện vật thể YOLOv8, và (3) Phương pháp ứng dụng Mô hình ngôn ngữ lớn (LLM) để tự động hóa công tác lập báo cáo tổng kết.

5.1 Giải pháp đồng bộ hóa Video và Dữ liệu cảm biến thời gian thực

5.1.1 Bài toán đặt ra

Như đã trình bày ở phần Khảo sát, một trong những khó khăn lớn nhất của đội ngũ vận hành ROV là việc phải đối chiếu thủ công giữa đoạn video ghi hình và tệp nhật ký thông số cảm biến (CSV). Do camera ghi hình và bo mạch cảm biến thường hoạt động độc lập (đôi khi không cùng khởi động vào một thời điểm chính xác), hệ quy chiếu thời gian của hai luồng dữ liệu này bị lệch. Bài toán đặt ra là làm thế nào để khi người dùng tương tác (Play/Pause/Seek) trên video, thanh đánh dấu trên biểu đồ cảm biến (áp suất, nhiệt độ, độ sâu) phải lập tức nhảy đến đúng thời điểm đo đạc tương ứng với khung hình đang phát, và ngược lại.

5.1.2 Giải pháp hiện thực

Để giải quyết vấn đề này, giải pháp đề xuất sử dụng thuật toán ánh xạ thời gian dựa trên độ lệch gốc (Time Offset Mapping).

- **Bước 1: Chuẩn hóa dữ liệu đầu vào.** Hệ thống gán trường `recordedAt` (thời điểm bắt đầu ghi hình theo chuẩn ISO 8601) cho video tải lên theo cơ chế ưu tiên ba cấp: (1) nếu lô tải lên kèm tệp `trip.json` (manifest xuất tự động từ ROV), hệ thống đọc `sessionStart + asset.start_ms` để suy ra `recordedAt` với độ chính xác mili-giây; (2) nếu không có manifest, hệ thống tự động parse từ tên file theo mẫu `_YYYYMMDD_HHMMSS`; (3) nếu cả hai đều không có, `recordedAt` được đặt là `null` và Operator có thể nhập thủ công qua giao diện chỉnh sửa. Tệp CSV cảm biến bắt buộc phải chứa cột *timestamp* ở mỗi dòng.
- **Bước 2: Tính toán độ lệch (Offset).** Khi lướt lặn được tải xuống trình duyệt, hệ thống tính toán sự chênh lệch (tính bằng giây) giữa thời điểm ghi dòng dữ liệu cảm biến đầu tiên và thời điểm bắt đầu video.

- **Bước 3: Cập nhật trạng thái liên tục.** Tại Frontend (sử dụng ReactJS), sự kiện `timeupdate` của phần tử HTML5 `<video>` được lắng nghe để cập nhật trạng thái `currentTime` vào React state mỗi khi video phát sang khung hình mới. Giá trị này được kết hợp với `media.recordedAt` để tính `chartTimestamp = recordedAt + currentTime × 1000` (đổi sang mili-giây UTC).
- **Bước 4: Ánh xạ sang dữ liệu cảm biến.** Dữ liệu cảm biến là một mảng được sắp xếp tăng dần theo `timestamp`. Hệ thống tìm điểm gần nhất với `chartTimestamp` và truyền vào thành phần `<ReferenceLine x={chartTimestamp}>` của thư viện `Recharts` để vẽ đường giống đồng bộ (Synchronized Reference Line) di chuyển trên biểu đồ theo thời gian thực.

5.1.3 Kết quả đạt được

Giải pháp đã tạo ra một giao diện *Cockpit* đồng bộ mượt mà theo chiều video → biểu đồ. Khi video đang phát, thanh đỏ `ReferenceLine` di chuyển liên tục trên biểu đồ cảm biến, cho phép Operator quan sát ngay giá trị nhiệt độ, áp suất, độ sâu tại đúng khoảnh khắc đang diễn ra trên video mà không cần đổi chiều thủ công. Cơ chế đồng bộ này loại bỏ hoàn toàn việc tra cứu thủ công giữa video và tệp CSV, giúp quá trình phân tích hậu kỳ nhanh hơn đáng kể so với phương pháp truyền thống.

5.2 Kiến trúc xử lý bất đồng bộ tích hợp mô hình nhận diện YOLOv8

5.2.1 Bài toán đặt ra

Nhận diện vật thể trong video (như cá, chướng ngại vật, hoặc vết nứt đường ống ngầm) là yêu cầu thiết yếu. Tuy nhiên, việc chạy suy luận mô hình học sâu như YOLOv8 trên các tệp video độ phân giải cao tiêu tốn lượng lớn tài nguyên CPU/GPU và mất nhiều thời gian (thường từ vài phút đến hàng giờ tùy độ dài). Nếu triển khai trực tiếp luồng xử lý này trên máy chủ Node.js chính, vòng lặp sự kiện (Event Loop) của Node sẽ bị chặn (block), dẫn đến việc toàn bộ hệ thống web bị "đóng băng" đối với các người dùng khác.

5.2.2 Giải pháp hiện thực

Giải pháp đề ra là xây dựng một kiến trúc tách rời (Decoupled Architecture) gồm ba thành phần phối hợp:

- **Node.js Express (HTTP endpoint):** Khi người dùng nhấn “Phân tích AI”, endpoint `POST/media/:id/analyze` đặt `analysisStatus='pending'` cho media document, đóng gói `mediaId`, mô hình và ngưỡng confidence thành một Job rồi đẩy vào hàng đợi Bull (`mediaAnalysisQueue`,

lưu trên Redis). Hệ thống trả về HTTP 202 ngay lập tức mà không chờ xử lý.

- **Node.js Bull Worker (xử lý ngầm):** Worker chạy song song với Event Loop chính, lắng nghe hàng đợi. Khi có Job, Worker tạo presigned GET URL (TTL 10 phút) cho file trên S3, sau đó gọi `POST yolo-service:8000/detect` với URL đó kèm tham số model và confidence. YOLO Service (Python FastAPI) tải file từ S3, chạy suy luận YOLOv8, trả về mảng labels qua HTTP response thông thường. Worker nhận kết quả, lưu `media.labels+analysisStatus='done'` vào MongoDB, rồi gọi `notificationService.create()` để ghi thông báo vào DB và đẩy SSE.
- **Python FastAPI (YOLO Microservice):** Là dịch vụ HTTP thuần túy, không kết nối Redis hay MongoDB. Chỉ nhận `POST /detect`, tải file từ presigned URL, chạy YOLOv8 inference, và trả về JSON labels trong một response đồng bộ. Kiến trúc này tách biệt hoàn toàn: Node.js có thể thay thế YOLO bằng mô hình AI khác mà không cần đổi hàng đợi hay schema DB.

5.2.3 Kết quả đạt được

Kiến trúc này đảm bảo tính khả dụng (High Availability) tuyệt đối cho hệ thống web chính. Người dùng có thể đẩy cùng lúc nhiều video vào hàng đợi để phân tích ngầm và thoải mái chuyển sang các trang khác để làm việc. Kết quả nhận diện sau khi trả về sẽ tự động dính kèm (overlay) lên luồng phát video, khoanh vùng chính xác các vật thể dưới nước một cách tự động.

5.3 Hệ thống trích xuất bằng chứng (Evidence System)

5.3.1 Bài toán đặt ra

Khi Operator phát hiện vật thể quan trọng (mảnh vỡ ngầm, sinh vật biển hiếm gặp, vết nứt kết cấu) trong khi xem lại video, không có công cụ nào cho phép đánh dấu và lưu trữ phát hiện đó ngay lập tức. Cách xử lý thủ công thông thường là ghi lại dấu thời gian rồi sau đó cắt video bằng phần mềm bên ngoài — tốn thời gian và làm gián đoạn luồng xem xét. Bài toán đặt ra là xây dựng một cơ chế cho phép Operator trích xuất bằng chứng trực tiếp trong khi xem video mà không làm gián đoạn trải nghiệm xem, đồng thời cho phép phân tích AI riêng lẻ trên từng bằng chứng mà không cần phân tích lại toàn bộ video.

5.3.2 Giải pháp hiện thực

Giải pháp đề xuất xây dựng hệ thống bằng chứng với hai loại artefact độc lập:

- **Photo (Ảnh khung hình):** Khi Operator nhấn nút chụp ảnh, Canvas API của trình duyệt trích xuất khung hình hiện tại thành tệp PNG. Nếu chế độ hiển thị bounding box đang bật, tọa độ được ghi đè trực tiếp vào ảnh (*canvas burn-in*)

trước khi tải lên AWS S3. Tập PNG được lưu vào trường `imageS3Key` của document Snapshot.

- **Clip (Đoạn video):** Khi Operator bấm bắt đầu và kết thúc ghi clip, hệ thống chỉ lưu `startTime` và `endTime` (tính bằng giây trong video gốc), không cắt hay sao chép tập video vật lý. Đây là lựa chọn kỹ thuật quan trọng: tránh hoàn toàn chi phí băng thông và không gian lưu trữ phát sinh từ việc tạo bản sao video.

Mỗi bằng chứng được lưu vào collection `snapshots` riêng biệt (không lẫn vào collection `media`), với trường `parentMediaId` liên kết đến video gốc. Để phân tích AI riêng từng bằng chứng, hệ thống cho phép gọi `POST/snapshots/:id/analyze`: với Photo, YOLO Service chạy predict trên ảnh PNG; với Clip, YOLO Service nhận thêm `startTime/endTime` và chỉ xử lý các frame trong khoảng đó, giảm đáng kể thời gian xử lý so với phân tích toàn bộ video.

Giao diện EvidenceViewer (Hình 4.14) được hiển thị dưới dạng overlay inline — không phải cửa sổ popup riêng — bảo toàn ngữ cảnh xem của Operator. Với clip, trình phát video bị giới hạn trong khoảng $[t_{start}, t_{end}]$: khi phát đến `endTime`, video tự seek về `startTime` để phát vòng lặp.

5.3.3 Kết quả đạt được

Hệ thống bằng chứng cho phép ghi lại phát hiện trong thời gian thực mà không cần rời khỏi giao diện xem video. Đặc biệt, nhờ thiết kế không cắt tập vật lý đối với Clip, chi phí lưu trữ S3 chỉ tăng theo số lượng ảnh PNG thumbnail — không phát sinh thêm từ các đoạn clip dài. Bằng chứng có thể được phân tích AI độc lập, tạo ra vòng lặp kiểm tra nhanh: *xem video* → *phát hiện* → *trích xuất* → *phân tích* → *xác nhận* mà không cần phân tích lại toàn bộ file video dài.

5.4 Phát hiện bất thường cảm biến tự động (Z-Score)

5.4.1 Bài toán đặt ra

Mỗi lượt lặn ROV tạo ra hàng nghìn dòng dữ liệu cảm biến liên tục (nhiệt độ, áp suất, độ sâu, điện áp...). Khi xảy ra sự cố — chẳng hạn thiết bị lặn quá sâu, rò rỉ nước vào khoang điện, hoặc va chạm làm thay đổi đột ngột áp suất — các điểm bất thường này bị chôn vùi trong hàng nghìn dòng dữ liệu bình thường. Việc tìm kiếm thủ công bằng mắt trên biểu đồ là không thực tế, nhất là với các bộ dữ liệu dài. Bài toán đặt ra là tự động phát hiện và gắn cờ các điểm đo bất thường ngay tại backend, không cần dữ liệu huấn luyện từ trước.

5.4.2 Giải pháp hiện thực

Giải pháp triển khai thuật toán Z-Score, một phương pháp thống kê kinh điển để phát hiện outlier không cần nhãn (unsupervised). Với mỗi metric trong bộ dữ liệu

cảm biến, Z-Score của từng điểm đo được tính theo công thức:

$$z_i = \frac{x_i - \mu}{\sigma} \quad (5.1)$$

trong đó x_i là giá trị đo tại thời điểm i , μ là giá trị trung bình và σ là độ lệch chuẩn của toàn bộ chuỗi. Điểm đo được đánh dấu là bất thường khi $|z_i| > 2,5$ — tức là vượt quá 2,5 lần độ lệch chuẩn so với giá trị trung bình. Ngưỡng 2,5 được chọn thay vì giá trị thông thường 2,0 hoặc 3,0 nhằm cân bằng giữa độ nhạy (sensitivity) và tỉ lệ cảnh báo nhầm (false positive rate) phù hợp với đặc thù dữ liệu cảm biến môi trường thay đổi chậm.

Thuật toán được áp dụng song song cho 7 metric: depth, temp, temperature, pressure, voltage, battery_percent và humidity. Kết quả trả về cùng với dữ liệu cảm biến:

- stats: {min, max, avg} cho mỗi metric
- anomalies: mảng các điểm bất thường, mỗi phần tử gồm {index, metric, value, zScore, timestamp}

So sánh với các giải pháp thay thế: phương pháp IQR (Interquartile Range) kém nhạy với spike nhọn ngắn hạn — đặc trưng của sự cố kỹ thuật; Isolation Forest đòi hỏi dữ liệu huấn luyện được gán nhãn, không khả thi trong bối cảnh hệ thống chưa có dataset lịch sử. Z-Score được chọn vì không cần dữ liệu huấn luyện, tính toán đơn giản, đủ nhanh để chạy inline tại endpoint API.

Tại giao diện, các điểm bất thường được hiển thị bằng dot đỏ kích thước lớn hơn trên AreaChart (Hình 4.11), kèm theo một panel "Anomalies Detected" liệt kê từng điểm cùng giá trị đo và chỉ số Z-Score tương ứng.

5.4.3 Kết quả đạt được

Thuật toán phát hiện thành công các spike bất thường trong dữ liệu kiểm thử (file sensor-sample-full.csv có 2 điểm bất thường được thiết kế sẵn), hiển thị đúng trên biểu đồ mà không ảnh hưởng đến hiệu năng API. Giải pháp này loại bỏ hoàn toàn gánh nặng tìm kiếm thủ công, chuyển từ quy trình “xem toàn bộ → phát hiện thủ công” sang “mở trang → các điểm nghi vấn đã được đánh dấu sẵn”, giúp Operator tập trung ngay vào những khoảnh khắc quan trọng nhất trong hàng giờ dữ liệu.

5.5 Ứng dụng LLM tự động hóa báo cáo tổng kết nhiệm vụ

5.5.1 Bài toán đặt ra

Kết thúc một Project khảo sát với nhiều Trip (lần lặn), ban quản lý thường yêu cầu một báo cáo tóm tắt tình hình tổng thể. Việc thủ công tổng hợp số liệu (số Trip, trạng thái từng Trip, tọa độ địa lý ghi nhận, các điểm cảm biến bất thường, danh sách vật thể phát hiện) ra văn bản rất khô khan, dễ thiếu sót và tốn hàng giờ đồng hồ rà soát.

5.5.2 Giải pháp hiện thực

Đề án ứng dụng Mô hình ngôn ngữ lớn (Large Language Model) - cụ thể là Gemini 2.5 Flash thông qua API của Google - để sinh văn bản tự động (Natural Language Generation). Giải pháp được thực hiện qua các bước:

- **Tổng hợp siêu dữ liệu:** Khi người dùng bấm "Generate Summary", Backend gom toàn bộ dữ liệu Project từ MongoDB: tên, địa điểm, thời gian, danh sách Trip kèm trạng thái, số lượng media và các điểm bất thường cảm biến (đã phát hiện bởi Z-Score, ngưỡng $|z| > 2,5$). Nếu Trip có GPS, `locationName` từ Nominatim được đưa vào prompt thay cho tọa độ số.
- **Thiết kế Mẫu nhắc (Prompt Engineering):** Dữ liệu thô (JSON) sẽ được "bơm" vào một cấu trúc Prompt tiếng Việt chuẩn mực. Ví dụ: *"Dựa trên dữ liệu hệ thống đo đạc từ ROV: Chuyến đi tên là [X], diễn ra từ [Ngày A] đến [Ngày B], thực hiện [Y] lượt lặn. Đã phát hiện [Z] vật thể (chủ yếu là cá, san hô) và có 3 lần cảnh báo áp suất bất thường. Hãy viết một báo cáo chuyên nghiệp dành cho ban quản đốc, tóm tắt tình hình chuyến đi, kết quả khảo sát và khuyến nghị an toàn."*
- **Gọi API và Lưu trữ bất đồng bộ:** Yêu cầu sinh tóm tắt được đưa vào hàng đợi Bull (`ai-summary`) và trả về HTTP 202 tức thì — không block giao diện. Bull Worker gọi Gemini 2.5 Flash API với prompt đã soạn, nhận phản hồi văn bản song ngữ (Việt/Anh), lưu vào `project.aiSummary.vi` và `project.aiSummary.en`, rồi đẩy SSE thông báo đến frontend khi hoàn tất.

5.5.3 Kết quả đạt được

Giải pháp biến những hàng nghìn dòng dữ liệu thô vô hồn thành một văn bản báo cáo liền mạch, mang tính phân tích cao chỉ trong khoảng 5-10 giây. Tính năng này không chỉ khẳng định sự chuyển dịch từ số hóa dữ liệu (Digitization) sang thấu hiểu dữ liệu (Data Insights), mà còn tạo ra một giá trị gia tăng cực lớn cho quy trình nghiệp vụ thực địa của các doanh nghiệp khảo sát.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã xuất sắc hoàn thành mục tiêu thiết kế và xây dựng "Phân hệ quản lý thông tin nhiệm vụ ROV". Vượt ra ngoài giới hạn của một ứng dụng lưu trữ và quản lý video thông thường, hệ thống đã giải quyết thành công bài toán phân mảnh dữ liệu thực địa bằng cách cung cấp cơ chế đồng bộ hóa mượt mà giữa hình ảnh và các thông số môi trường (áp suất, độ sâu, nhiệt độ) theo thời gian thực.

So với các giải pháp hiện hành trên thị trường (như QGroundControl hay BlueOS vốn chỉ tập trung vào khâu điều khiển phần cứng trực tiếp), điểm sáng lớn nhất của đồ án nằm ở khâu phân tích hậu kỳ (Post-mission Analytics) trên nền tảng web đa người dùng. Việc kiến trúc thành công dịch vụ vi mô (microservice) tích hợp mô hình nhận diện vật thể học sâu YOLOv8 kết hợp với cơ chế hàng đợi bất đồng bộ (Bull Queue) đã giúp tự động hóa quá trình dò tìm bất thường dưới nước mà không gây ảnh hưởng đến hiệu năng của máy chủ web chính. Cùng với đó, bước tiến táo bạo trong việc ứng dụng Mô hình ngôn ngữ lớn (Gemini 2.5 Flash) để tự động sinh báo cáo tổng kết đã giúp giảm thiểu hàng giờ đồng hồ làm việc thủ công của con người.

Nhìn chung, hệ thống đã chứng minh được tính khả thi trong việc số hóa và tự động hóa quy trình quản lý dữ liệu cho các đội vận hành phương tiện dưới nước. Tuy nhiên, hệ thống vẫn còn một số điểm chưa hoàn thiện, chủ yếu nằm ở việc chưa có các bài kiểm thử thực tế với lượng dữ liệu lên tới hàng Terabytes, cũng như mô hình YOLOv8 hiện tại mới chỉ dừng ở mức nhận diện các nhóm vật thể cơ bản được huấn luyện sẵn (Pre-trained), chưa phân loại được các sinh vật hoặc sự cố đường ống đặc thù.

6.2 Hướng phát triển

Dựa trên những kết quả đã đạt được, hệ thống có nhiều tiềm năng mở rộng trong tương lai để trở thành một nền tảng toàn diện hơn. Định hướng phát triển được chia làm hai mũi nhọn:

1. Hoàn thiện các chức năng hiện tại:

- **Tinh chỉnh mô hình học sâu (Fine-Tuning):** Tiến hành thu thập thêm các tập dữ liệu dưới nước chuyên biệt và huấn luyện lại mô hình YOLOv8 để có khả năng nhận dạng chính xác các cấu kiện kỹ thuật (vết nứt cáp ngầm, ăn mòn kim loại) hoặc các loài sinh vật biển đặc hữu.
- **Tối ưu hóa lưu trữ:** Triển khai cơ chế phân mảnh video (HLS/DASH) kết

hợp với thuật toán nén thông số cảm biến để tối ưu tốc độ tải trang cho người dùng có kết nối mạng kém.

2. Nghiên cứu các hướng đi mới:

- **Phân tích thời gian thực (Real-time Edge Computing):** Thay vì đợi nhiệm vụ kết thúc và tải dữ liệu lên web một cách thụ động, hướng tới việc tích hợp các mô-đun truyền tải vệ tinh (như Starlink) hoặc sóng âm dưới nước để đẩy trực tiếp luồng dữ liệu (Livestream) và cảnh báo từ ROV về hệ thống trung tâm ngay khi đang lặn.
- **Quản lý bầy đàn (Swarm Management):** Mở rộng kiến trúc cơ sở dữ liệu để hỗ trợ theo dõi, điều phối và vẽ bản đồ 3D từ dữ liệu của nhiều ROV cùng hoạt động đồng thời trong một khu vực địa lý lớn.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

hovy1993automated E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

peterson2007computer L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

NguyenThucHai N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

poesio2001discourse M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

knott1996data A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu

có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

BernersTim T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available: `ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[1] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] OpenJS Foundation, *Node.js — a JavaScript runtime built on Chrome's V8 JavaScript engine*, Accessed: 2026-06-22, 2024. [Online]. Available: <https://nodejs.org>

PHỤ LỤC

A. ĐẶC TẢ USE CASE

Phụ lục này bổ sung đặc tả chi tiết cho các Use Case cơ bản (nhóm chức năng CRUD và quản lý người dùng), vốn đóng vai trò nền tảng hỗ trợ cho các Use Case nghiệp vụ chính đã được trình bày trong Chương 2.

A.1 UC-B01: Đăng nhập bằng Email và Mật khẩu

Bảng A.1: Đặc tả Use Case UC-B01: Đăng nhập bằng Email/Mật khẩu

Mã Use Case	UC-B01
Tên	Đăng nhập bằng Email và Mật khẩu
Tác nhân	Viewer, Operator, Admin
Tiền điều kiện	Tác nhân có tài khoản đã được tạo trong hệ thống.
Luồng chính	(1) Tác nhân truy cập trang đăng nhập, nhập địa chỉ email và mật khẩu. (2) Hệ thống xác thực thông tin với cơ sở dữ liệu. (3) Hệ thống cấp phát Access Token (TTL 15 phút) và Refresh Token (TTL 7 ngày). (4) Tác nhân được chuyển hướng đến trang Dashboard.
Luồng phát sinh	Mật khẩu sai hoặc email không tồn tại → hệ thống trả lỗi 401; tài khoản bị vô hiệu hóa → trả lỗi 403; vượt quá 20 lần đăng nhập trong 15 phút → Rate Limiter trả lỗi 429.
Hậu điều kiện	Tác nhân được xác thực và có thể truy cập các tính năng theo quyền hạn vai trò.

A.2 UC-B02: Đăng nhập bằng tài khoản Google (OAuth 2.0)

Bảng A.2: Đặc tả Use Case UC-B02: Đăng nhập bằng Google OAuth 2.0

Mã Use Case	UC-B02
Tên	Đăng nhập bằng tài khoản Google (OAuth 2.0)
Tác nhân	Viewer, Operator, Admin
Tiền điều kiện	Tác nhân có tài khoản Google hợp lệ.

Luồng chính	(1) Tác nhân truy cập trang đăng nhập và chọn “<i>Continue with Google</i>”. (2) Hệ thống chuyển hướng sang trang xác thực của Google (OAuth consent screen). (3) Tác nhân xác nhận cấp quyền; Google trả về thông tin hồ sơ (email, tên, avatar). (4) Hệ thống kiểm tra email trong cơ sở dữ liệu: nếu là người dùng mới, tự động tạo tài khoản với vai trò mặc định <i>Viewer</i>; nếu đã tồn tại, liên kết <code>googleId</code>. (5) Hệ thống cấp phát JWT và chuyển hướng về <code>/auth/callback</code> trên frontend. (6) Tác nhân được đưa vào trang Dashboard.
Luồng phát sinh	Tác nhân hủy xác thực tại trang Google → hệ thống đưa về trang đăng nhập với thông báo lỗi; tài khoản bị vô hiệu hóa → trả lỗi 403.
Hậu điều kiện	Tác nhân đăng nhập thành công; tài khoản Google liên kết với hệ thống; tab “ <i>Đổi mật khẩu</i> ” bị ẩn trên Profile vì Google user không có mật khẩu.

A.3 UC-B03: Quản lý Thiết bị ROV

Bảng A.3: Đặc tả Use Case UC-B03: Quản lý Thiết bị ROV

Mã Use Case	UC-B03
Tên	Quản lý Thiết bị ROV (CRUD)
Tác nhân	Admin (xóa), Operator/Admin (tạo/sửa), Viewer (xem)
Tiền điều kiện	Tác nhân đã đăng nhập.
Luồng chính	(1) Tác nhân truy cập trang <i>ROV Registry</i>; hệ thống hiển thị danh sách thiết bị. (2) Operator/Admin chọn “<i>Thêm ROV mới</i>”, nhập tên, số serial, mô tả. (3) Hệ thống lưu document ROV vào MongoDB, hiển thị trong danh sách. (4) Tác nhân có thể chỉnh sửa thông tin ROV (Operator/Admin) hoặc xóa (Admin). (5) Admin xóa ROV: hệ thống ghi nhật ký (Audit Log) và cập nhật thống kê.

Luồng phát sinh	Xóa ROV đang có Project liên kết → hệ thống vẫn cho phép xóa; các Project cũ vẫn giữ tham chiếu. Operator cố xóa → hệ thống trả lỗi 403.
Hậu điều kiện	Thông tin ROV được cập nhật trong hệ thống; hành động xóa được ghi vào Audit Log.

A.4 UC-B04: Tạo và Quản lý Chuyến khảo sát (Project)

Bảng A.4: Đặc tả Use Case UC-B04: Quản lý Chuyến khảo sát

Mã Use Case	UC-B04
Tên	Tạo và Quản lý Chuyến khảo sát (Project CRUD)
Tác nhân	Operator, Admin
Tiền điều kiện	Tác nhân đã đăng nhập với quyền Operator hoặc Admin; ít nhất một thiết bị ROV đã tồn tại.
Luồng chính	(1) Tác nhân chọn “<i>Tạo Project mới</i>”, nhập tên, chọn ROV, địa điểm và thời gian bắt đầu dự kiến. (2) Hệ thống tạo document Project với trạng thái khởi tạo <i>planned</i>. (3) Tác nhân chuyển trạng thái sang <i>ongoing</i> khi bắt đầu thu thập dữ liệu thực địa. (4) Sau khi hoàn tất, tác nhân chuyển sang <i>completed</i> để mở khóa tính năng AI Summary. (5) Admin có thể xóa Project; hệ thống cascade xóa toàn bộ Trip, Media, SensorData liên quan.
Luồng phát sinh	Xóa Project đang có Media trên S3 → hệ thống cascade xóa file S3 trước khi xóa document; thao tác không thể hoàn tác.
Hậu điều kiện	Project và toàn bộ dữ liệu con được quản lý nhất quán; mọi thay đổi trạng thái được ghi Audit Log.

A.5 UC-B05: Tạo Lướt lặn trong Chuyến khảo sát

Bảng A.5: Đặc tả Use Case UC-B05: Tạo Lướt lặn (Trip)

Mã Use Case	UC-B05
Tên	Tạo Lướt lặn trong Chuyến khảo sát
Tác nhân	Operator, Admin
Tiền điều kiện	Chuyến khảo sát (Project) đã tồn tại.

Luồng chính	(1) Tác nhân truy cập ProjectDetailPage, chọn “<i>Thêm Trip mới</i>”. (2) Hệ thống hiển thị form nhập tên, mô tả lượt lặn. (3) Tác nhân xác nhận; hệ thống tạo document Trip với trạng thái <i>pending</i> và liên kết với Project. (4) Khi bắt đầu lặn thực tế, tác nhân chuyển trạng thái sang <i>running</i>. (5) Sau khi kết thúc và tải dữ liệu, tác nhân đánh dấu <i>done</i> hoặc <i>failed</i>. (6) Hệ thống gửi thông báo SSE đến tác nhân tạo Trip khi trạng thái thay đổi thành <i>done</i> hoặc <i>failed</i>.
Luồng phát sinh	Xóa Trip → cascade xóa SensorData, DVLDData, SonarFile, Media (S3 + DB), Snapshots (S3 + DB).
Hậu điều kiện	Trip được khởi tạo trong hệ thống, sẵn sàng tiếp nhận dữ liệu tải lên.

A.6 UC-B06: Tải lên Tập phương tiện (Media Upload)

Bảng A.6: Đặc tả Use Case UC-B06: Tải lên Tập phương tiện qua Presigned URL

Mã Use Case	UC-B06
Tên	Tải lên Tập phương tiện (Media Upload qua AWS S3 Presigned URL)
Tác nhân	Operator, Admin
Tiền điều kiện	Lượt lặn (Trip) đã tồn tại; tác nhân có quyền Operator/Admin.

Luồng chính	(1) Tác nhân chọn tệp Video hoặc Ảnh trong giao diện Upload. (2) Frontend gửi POST /media/upload-url {filename, type, size} đến backend; hệ thống trả về presignedUrl và mediaId. (3) Frontend tải file trực tiếp lên AWS S3 bằng PUT presignedUrl (không qua backend, không tốn băng thông server). (4) Frontend gửi POST /media/:id/confirm; hệ thống cập nhật media.status = 'ready' và áp dụng cơ chế ưu tiên: nếu recordedAt đã được gán từ trip_master.json (bước tải lên lô) thì giữ nguyên; ngược lại, tự động parse từ tên file nếu có mẫu _YYYYMMDD_HHMMSS; nếu không, recordedAt là null. (5) File xuất hiện trong MediaGallery của lượt lặn.
Luồng phát sinh	Bước PUT S3 thất bại → tác nhân không gọi confirm, document Media giữ trạng thái <i>pending</i> và được dọn dẹp sau; file lớn hơn 500MB → presigned URL từ chối.
Hậu điều kiện	Tệp phương tiện được lưu trên S3 và liên kết với Trip; analysisStatus = 'idle' (chưa phân tích YOLO).

A.7 UC-B07: Quản lý Người dùng (Admin)

Bảng A.7: Đặc tả Use Case UC-B07: Quản lý Người dùng

Mã Use Case	UC-B07
Tên	Quản lý Người dùng (User Management)
Tác nhân	Admin
Tiền điều kiện	Tác nhân phải có đặc quyền Admin.

Luồng chính	(1) Admin truy cập trang “<i>Quản lý Người dùng</i>”; hệ thống hiển thị danh sách tài khoản với trạng thái và vai trò. (2) Admin thay đổi vai trò người dùng (ví dụ: Viewer → Operator). (3) Admin kích hoạt hoặc vô hiệu hóa tài khoản. (4) Hệ thống lưu thay đổi, đẩy thông báo SSE đến tài khoản bị ảnh hưởng và ghi Audit Log. (5) Admin có thể thực hiện thao tác hàng loạt (Bulk): chọn nhiều tài khoản → đổi vai trò hoặc đổi trạng thái cùng lúc.
Luồng phát sinh	Admin tự vô hiệu hóa tài khoản của chính mình → hệ thống từ chối (self-protection) và hiển thị cảnh báo.
Hậu điều kiện	Phân quyền hệ thống được cập nhật an toàn; mọi thay đổi được ghi đầy đủ vào Audit Log.

A.8 UC-B08: Xem Dashboard tổng quan

Bảng A.8: Đặc tả Use Case UC-B08: Xem Dashboard tổng quan

Mã Use Case	UC-B08
Tên	Xem Dashboard tổng quan hệ thống
Tác nhân	Viewer, Operator, Admin
Tiền điều kiện	Tác nhân đã đăng nhập.
Luồng chính	(1) Tác nhân truy cập /dashboard; hệ thống gửi song song 7 aggregation query đến MongoDB. (2) Hệ thống trả về dữ liệu thống kê: tổng Projects, Trips đang chạy, ROVs hoạt động, tổng người dùng. (3) Giao diện hiển thị 4 thẻ KPI và 4 biểu đồ: Project Status (donut), Trips by Status (bar), ROV Utilization (horizontal bar), Activity Timeline 6 tháng gần nhất.
Luồng phát sinh	Mất kết nối MongoDB → hệ thống trả lỗi 500; giao diện hiển thị trạng thái lỗi thân thiện với nút thử lại.
Hậu điều kiện	Tác nhân có cái nhìn tổng quan về trạng thái hệ thống theo thời gian thực.